R & RStudio

Linguagem, Ambiente e Objetos

James Sampaio, PhD

R & RStudio

Linguagem, Ambiente e Objetos

- R
- RStudio
- Objetos
- Atalhos
- Vetores
- Matrizes
- Referências

CIÊNCIA DE DADOS



Esquentando

- O que vem à sua cabeça quando pensa nas palavras abaixo?

  ✓ Ciência

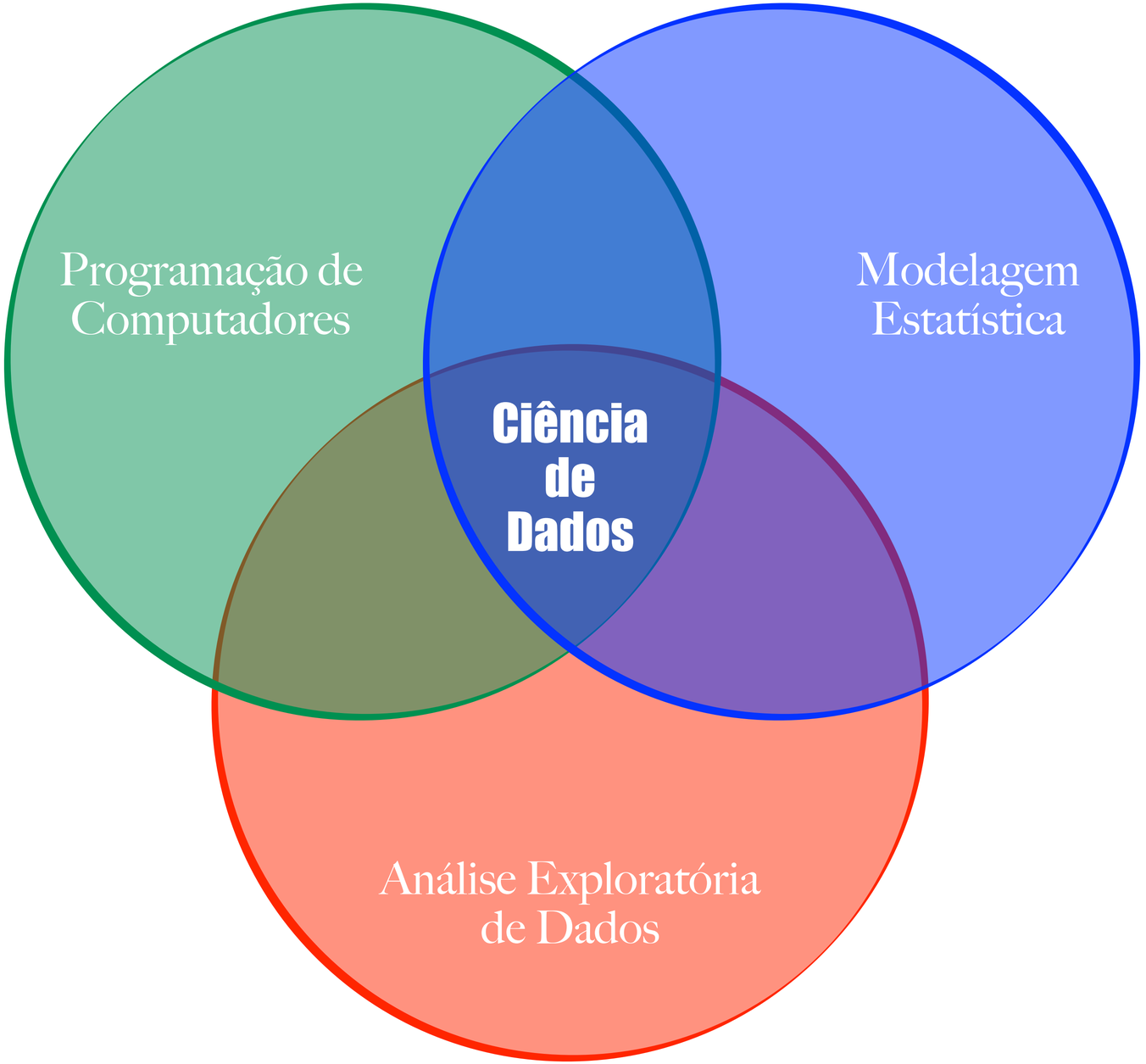
  ✓ Matemática

  ✓ Computação

  ✓ Estatística

  ✓ Ciência de Dados

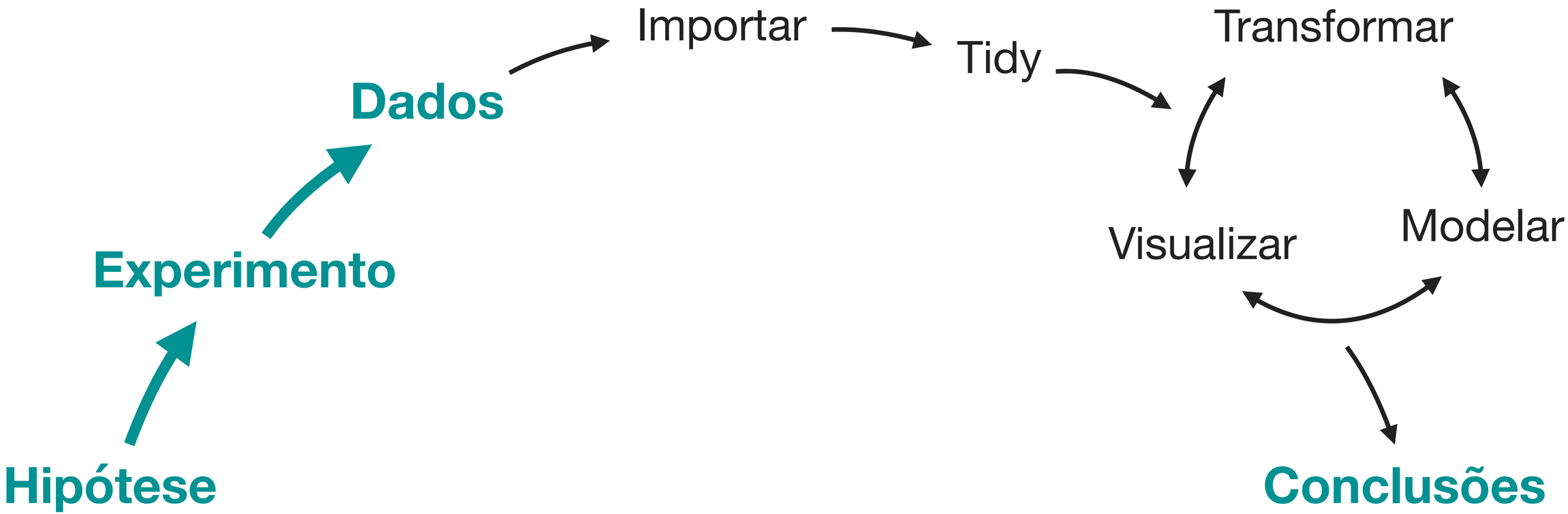
  Ciência a partir de dados não triviais ?


Programação de
Computadores
Modelagem
Estatística
Ciência
de
Dados
Análise Exploratória
de Dados

Ciência de Dados



Ciência de Dados Aplicada





R

Uma linguagem



R

Uma linguagem

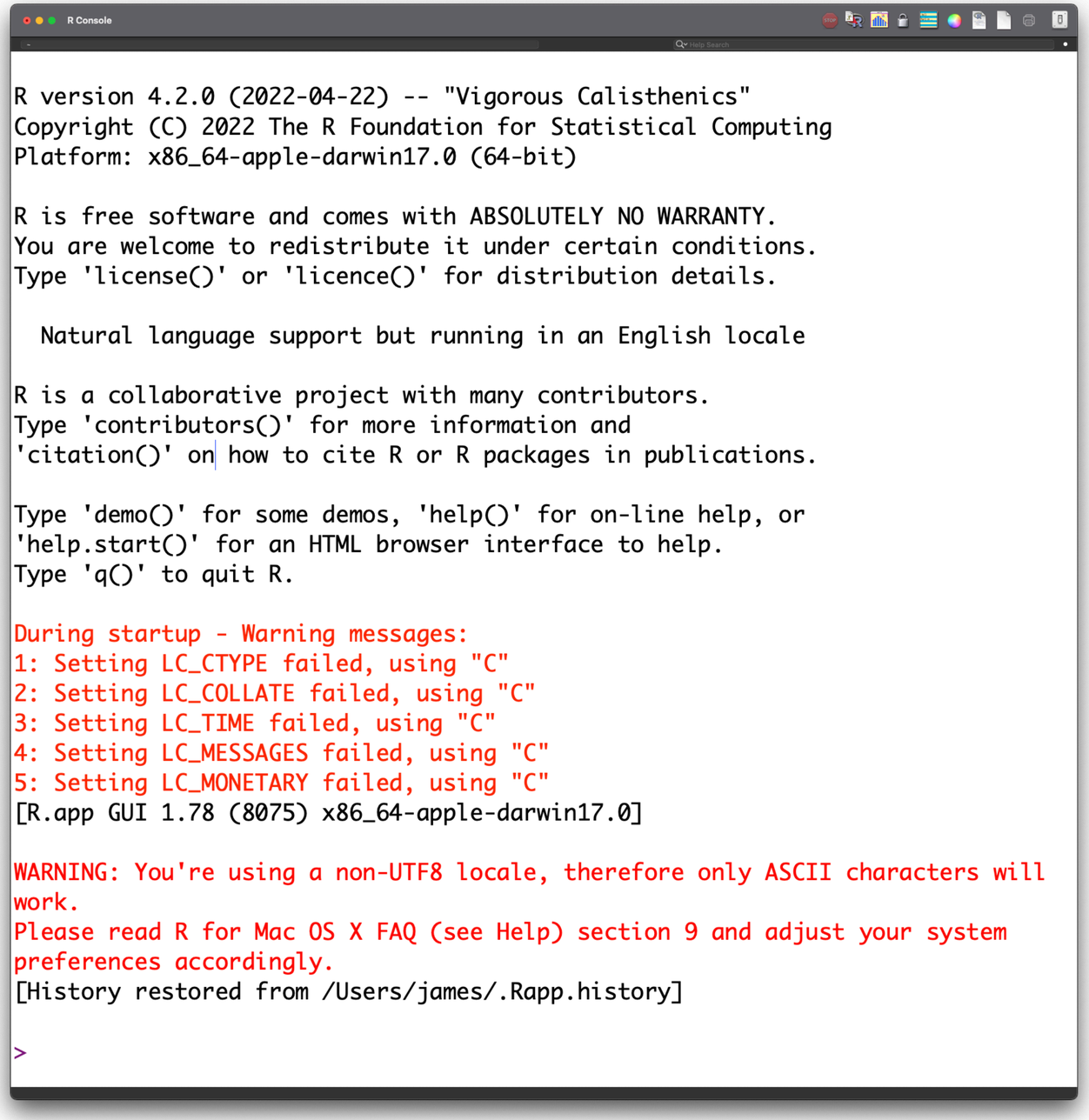
- Descendente da linguagem S [Bell Laboratories]
- Criada, em 1991, por Ross Ihaka e Robert Gentleman na Universidade de Auckland



R

Uma linguagem

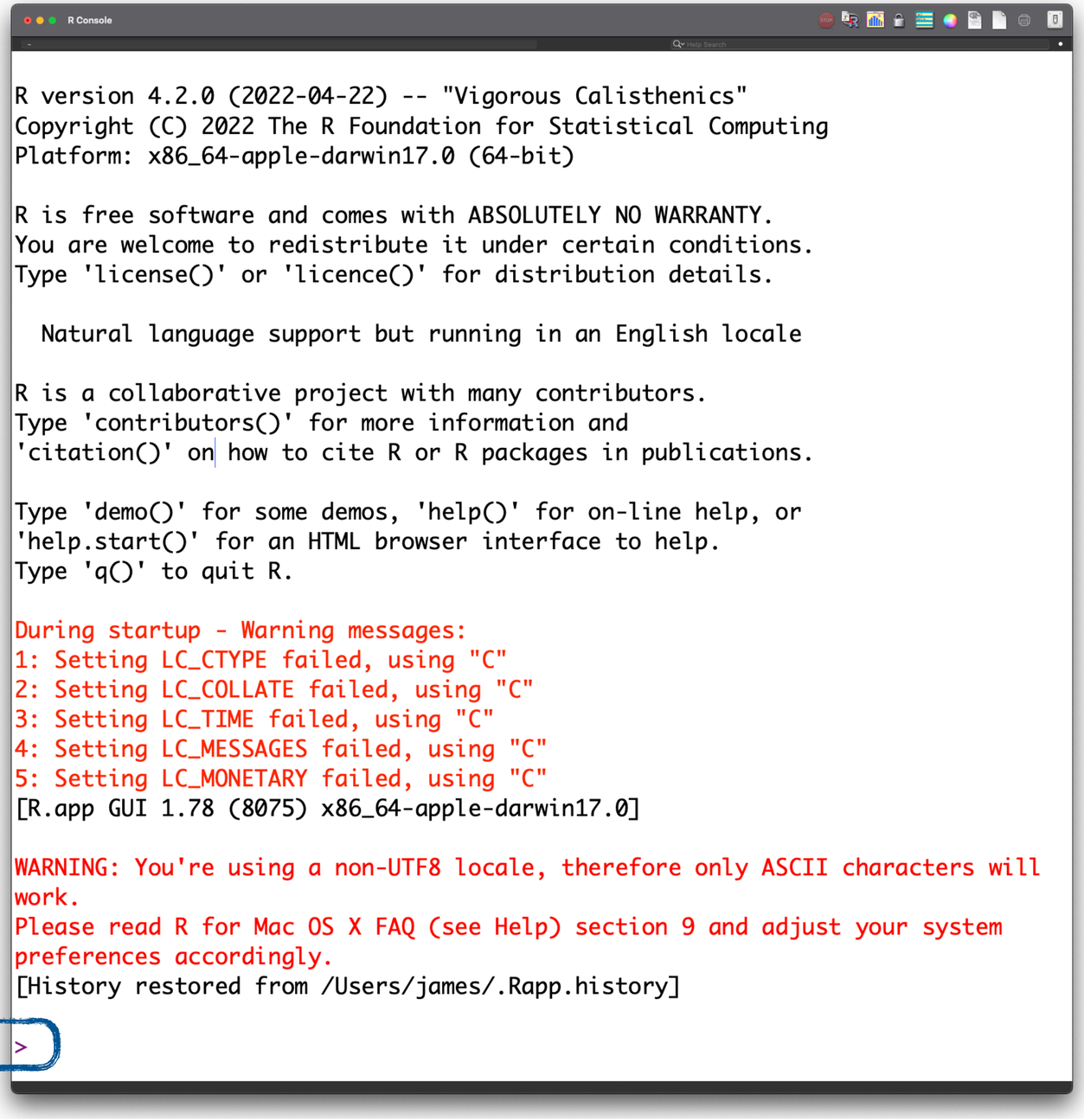
- Descendente da linguagem S [Bell Laboratories]
- Criada, em 1991, por Ross Ihaka e Robert Gentleman na Universidade de Auckland
- Gratuito e código-aberto
- r-project.org



R

Uma calculadora elegante

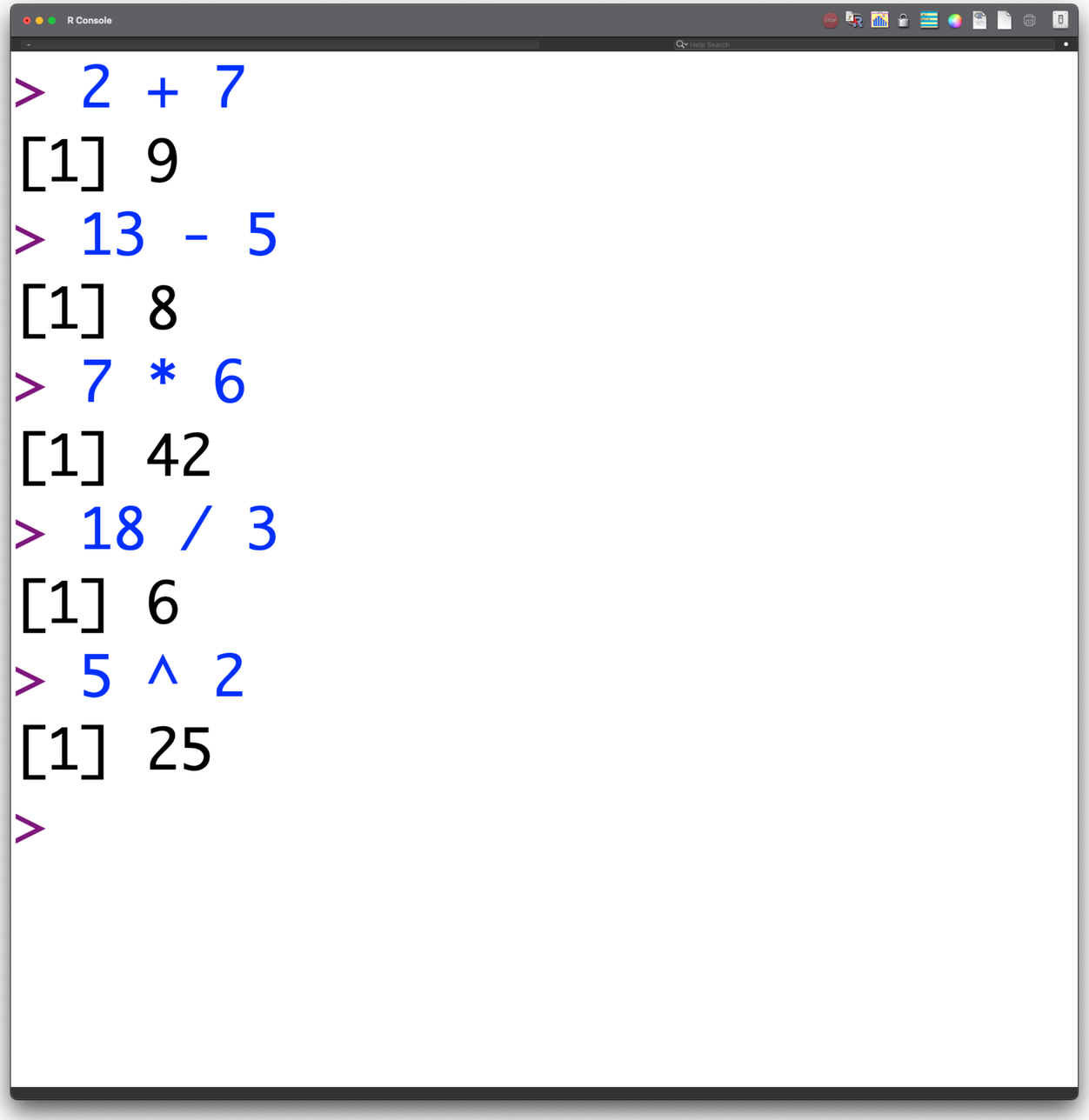
- Os comandos são executados no *Prompt*



R

Uma calculadora elegante

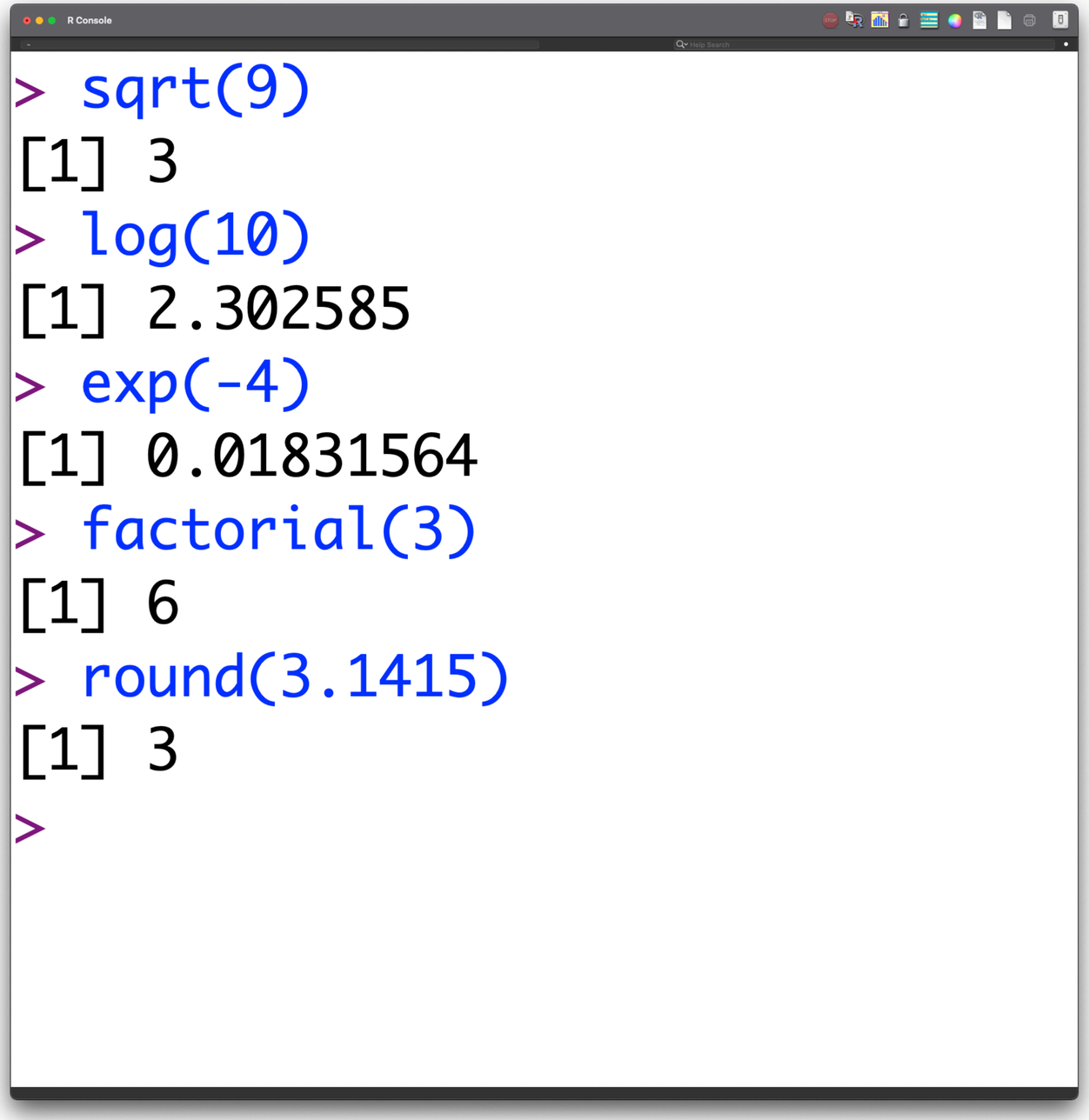
- Os comandos são executados no *Prompt*
- Operações matemáticas básicas são facilmente realizadas



R

Uma calculadora elegante

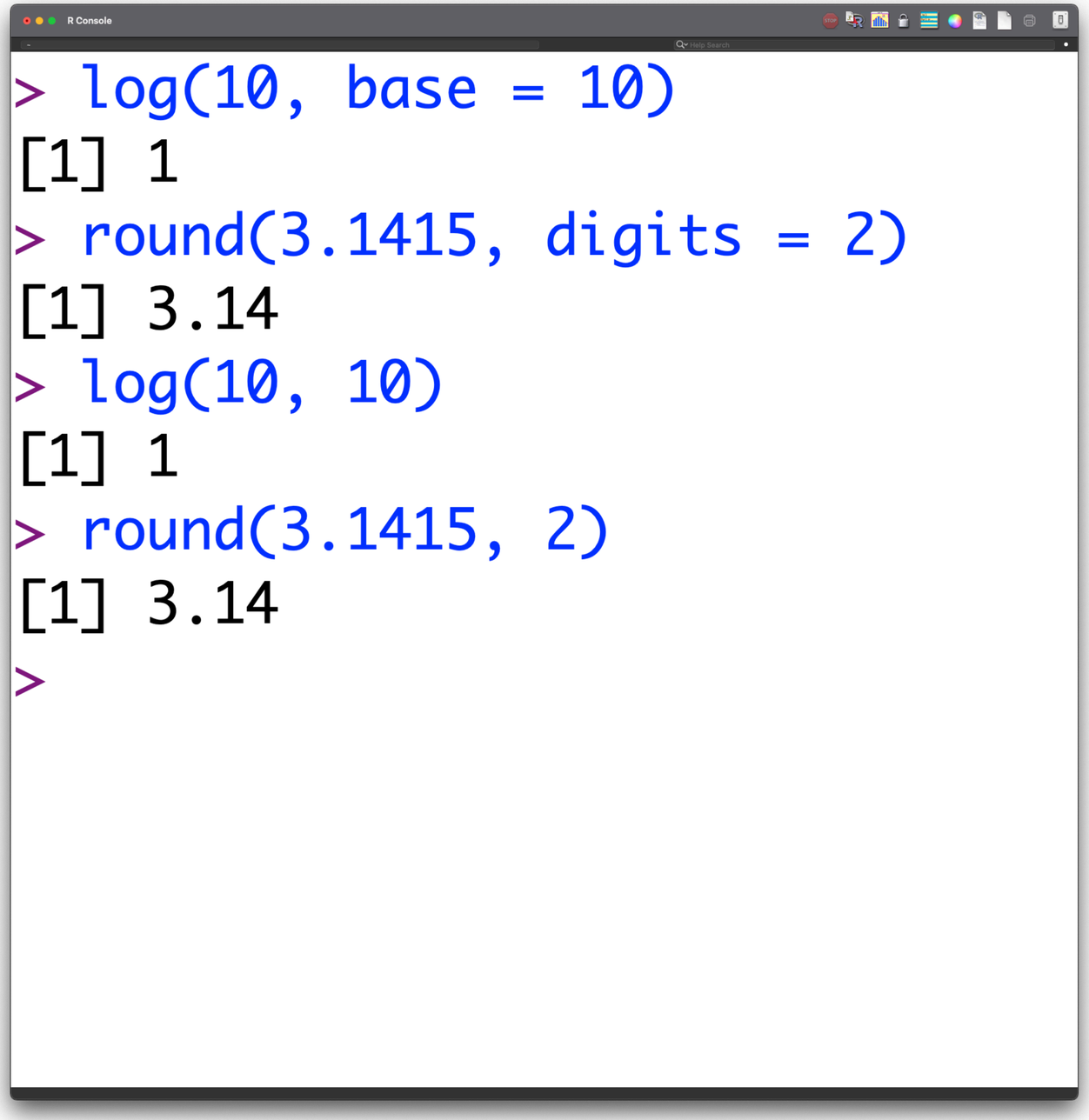
- Os comandos são executados no *Prompt*
- Operações matemáticas básicas são facilmente realizadas
- Funções, das mais diversas, são acessíveis de maneira simples



R

Uma calculadora elegante

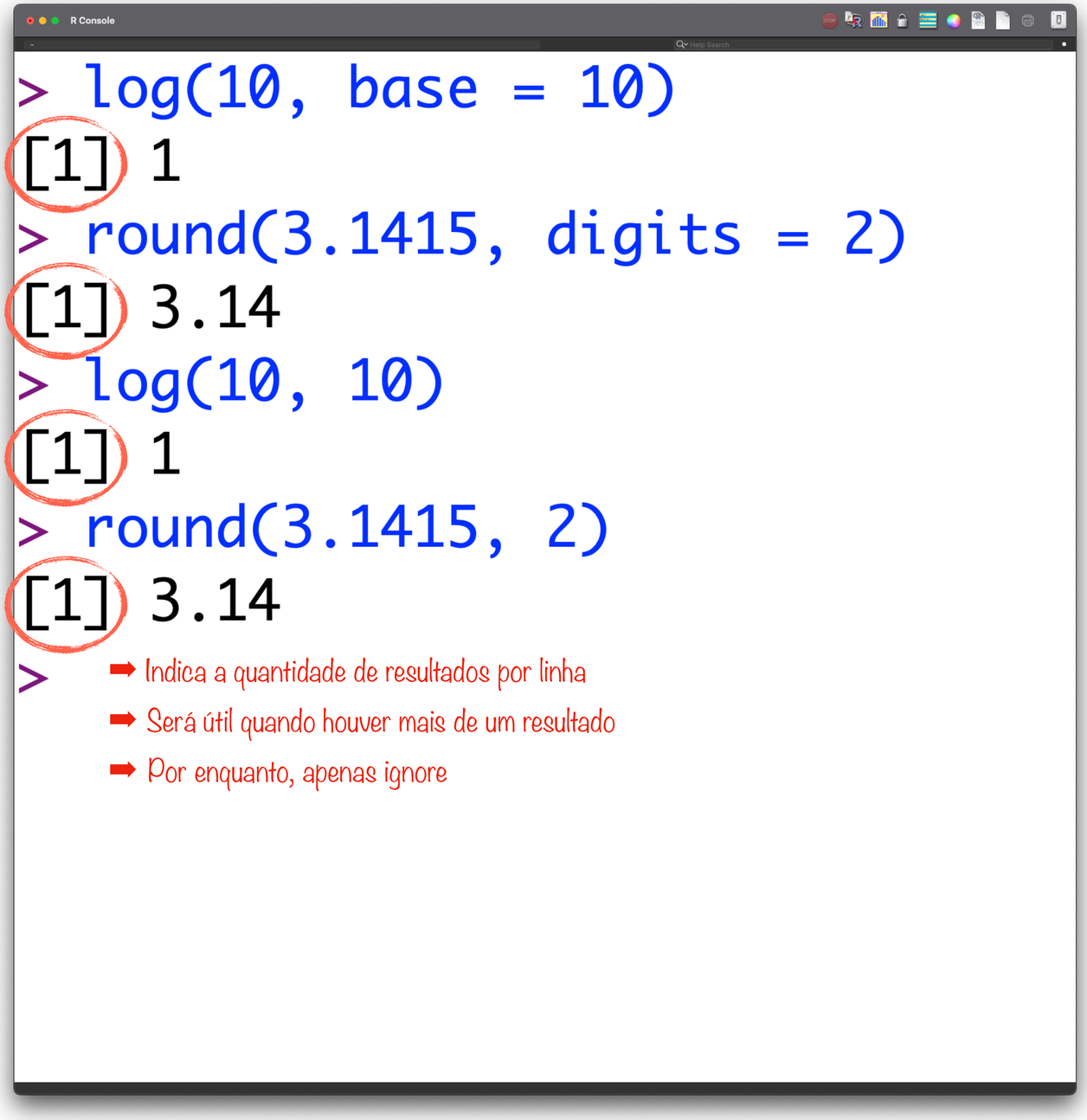
- Os comandos são executados no *Prompt*
- Operações matemáticas básicas são facilmente realizadas
- Funções, das mais diversas, são acessíveis de maneira simples



R

Uma calculadora elegante

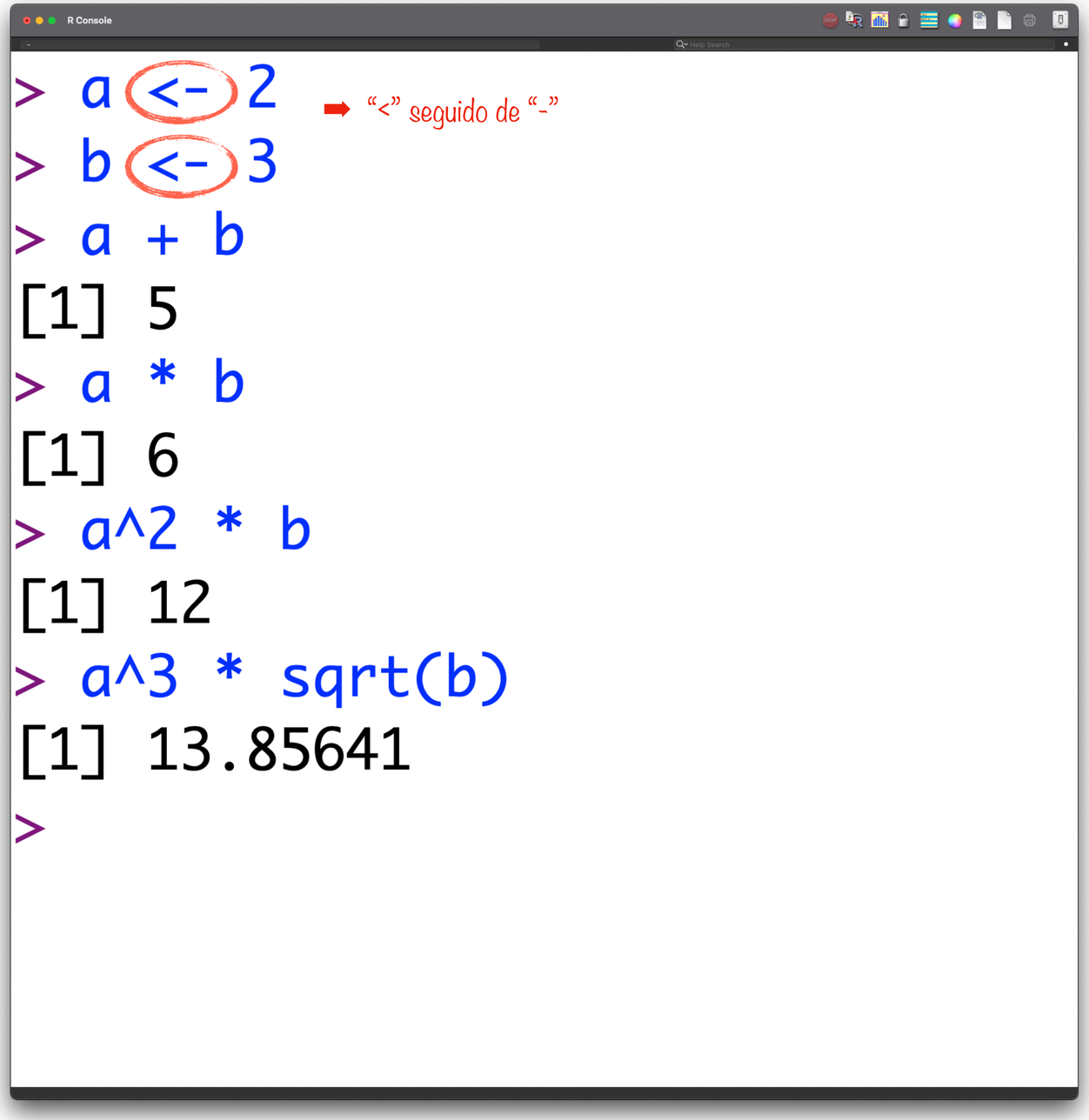
- Os comandos são executados no *Prompt*
- Operações matemáticas básicas são facilmente realizadas
- Funções, das mais diversas, são acessíveis de maneira simples



R

Uma calculadora elegante

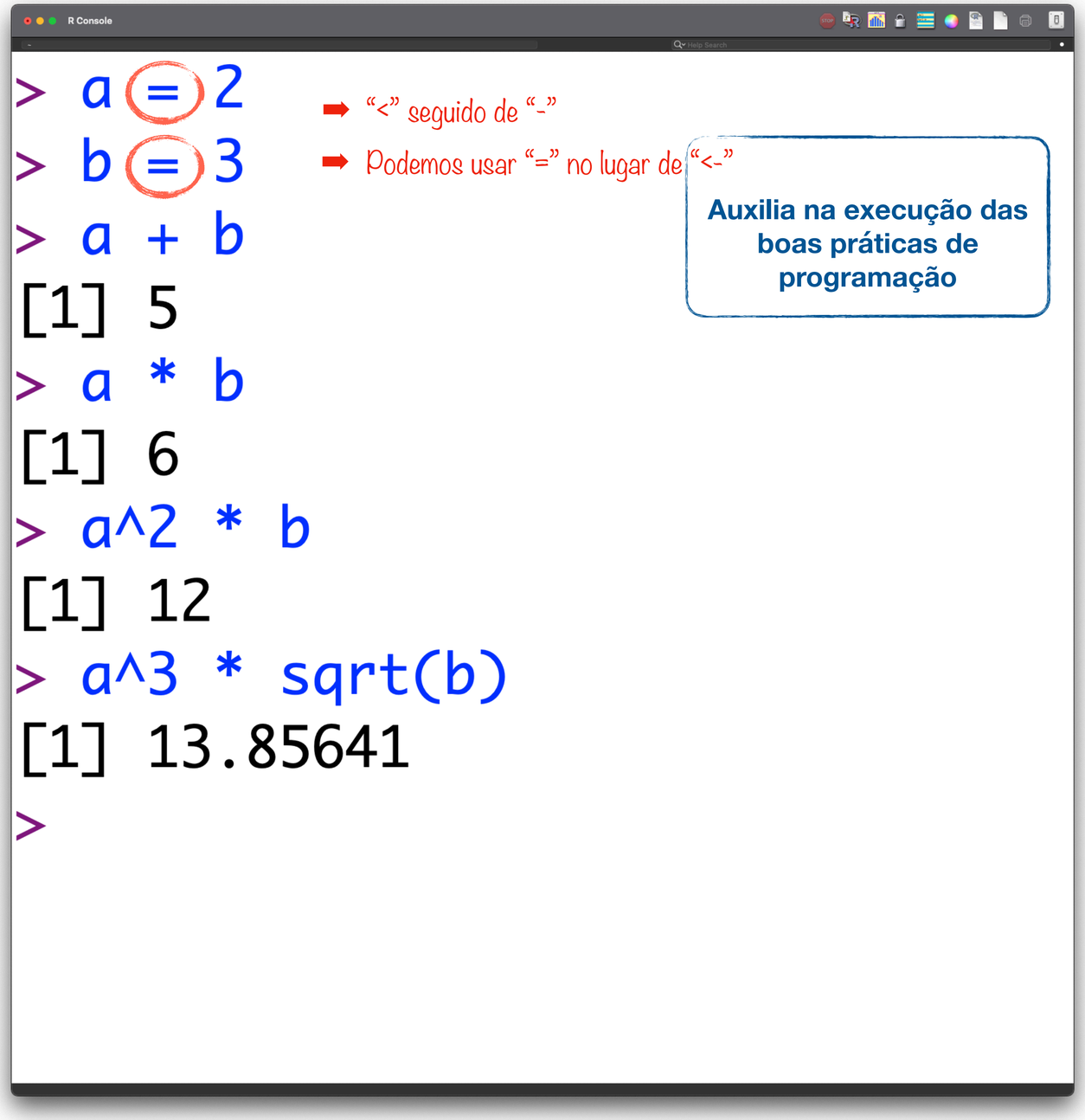
- Os comandos são executados no *Prompt*
- Operações matemáticas básicas são facilmente realizadas
- Funções, das mais diversas, são acessíveis de maneira simples
- Podemos atribuir valores a *objetos* e realizar cálculos algébricos



R

Uma calculadora elegante

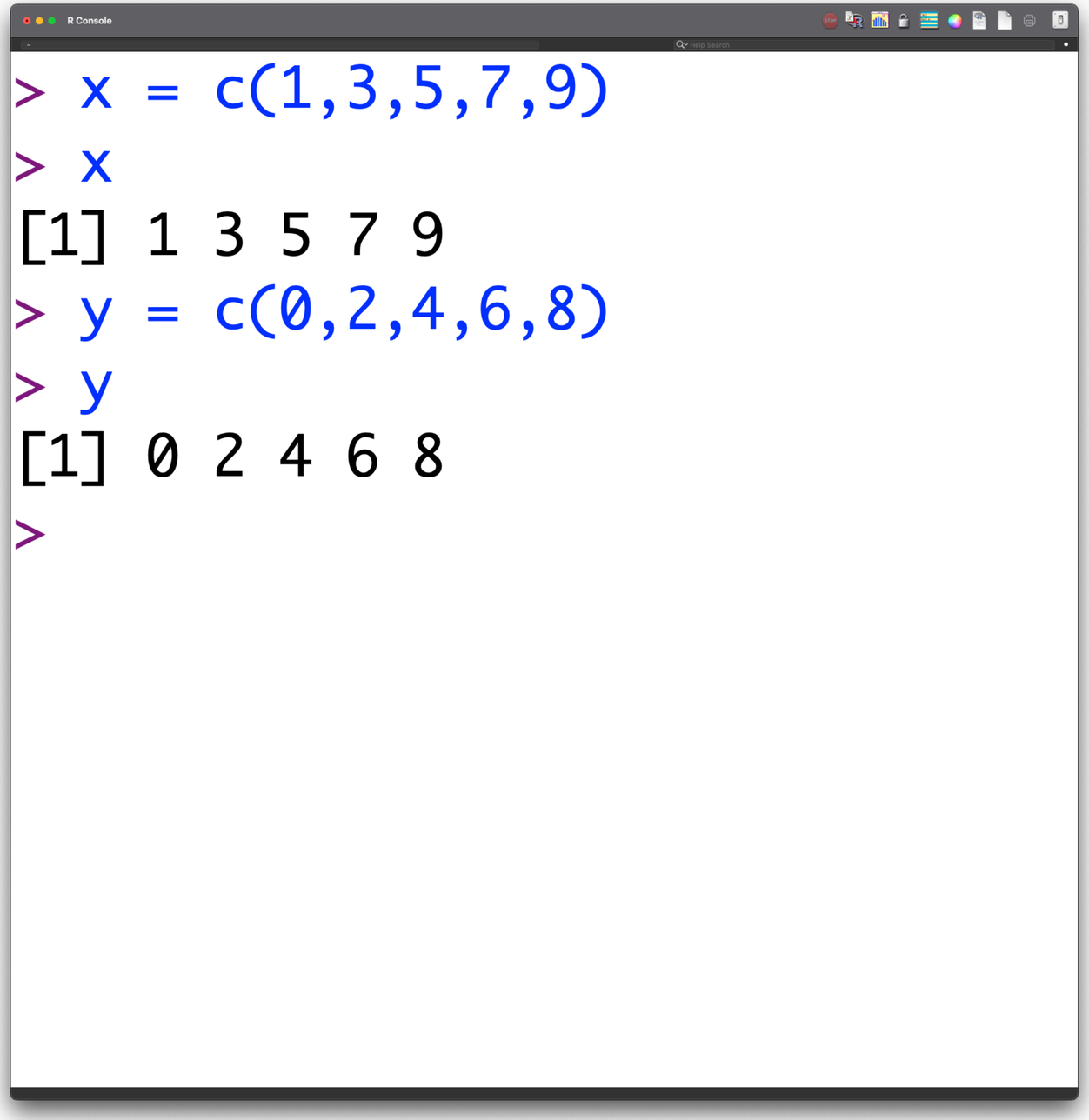
- Os comandos são executados no *Prompt*
- Operações matemáticas básicas são facilmente realizadas
- Funções, das mais diversas, são acessíveis de maneira simples
- Podemos atribuir valores a *objetos* e realizar cálculos algébricos



R

Uma calculadora elegante

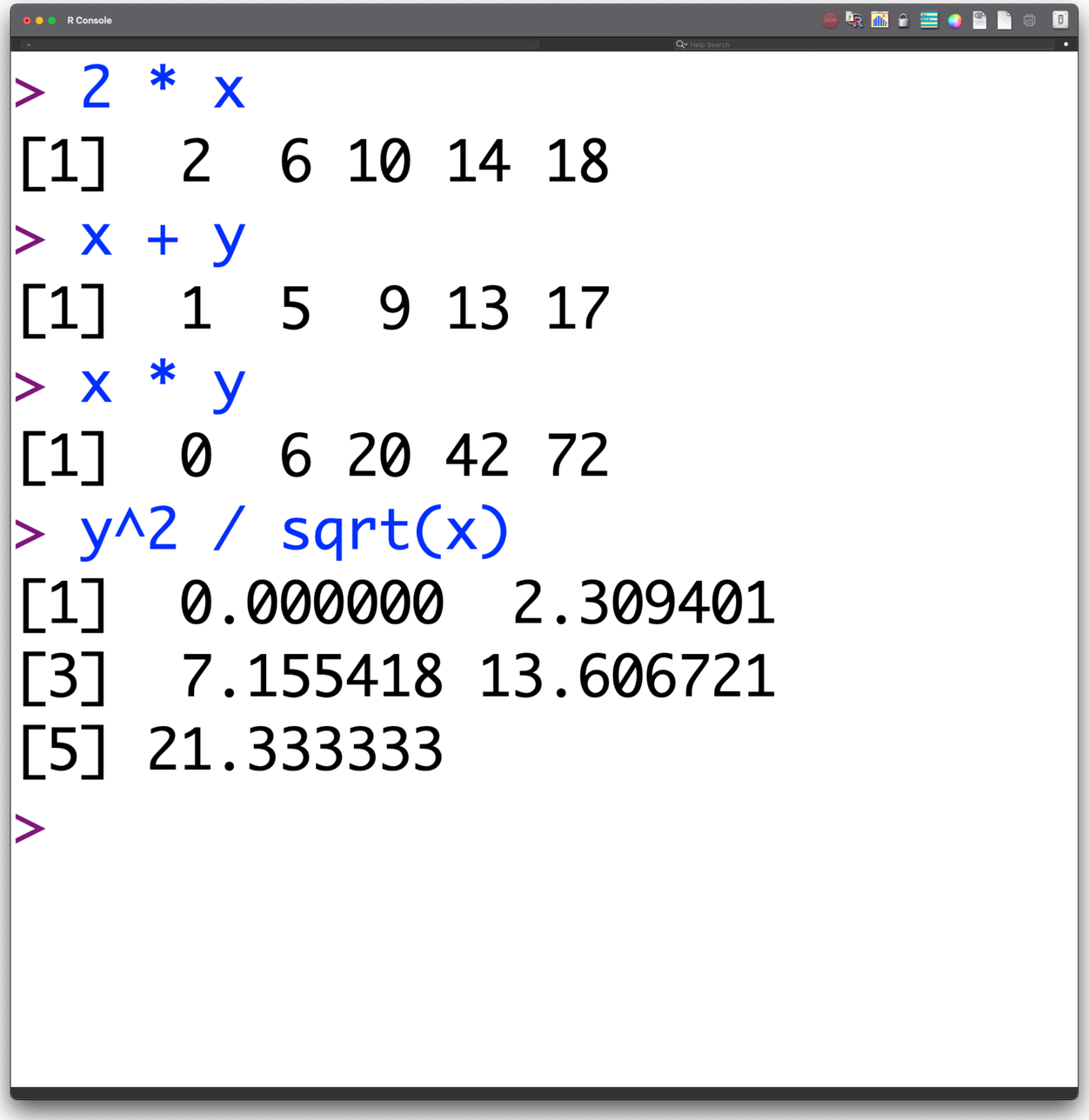
- Podemos salvar vetores em objetos; basta colocar os valores entre c( )



R

Uma calculadora elegante

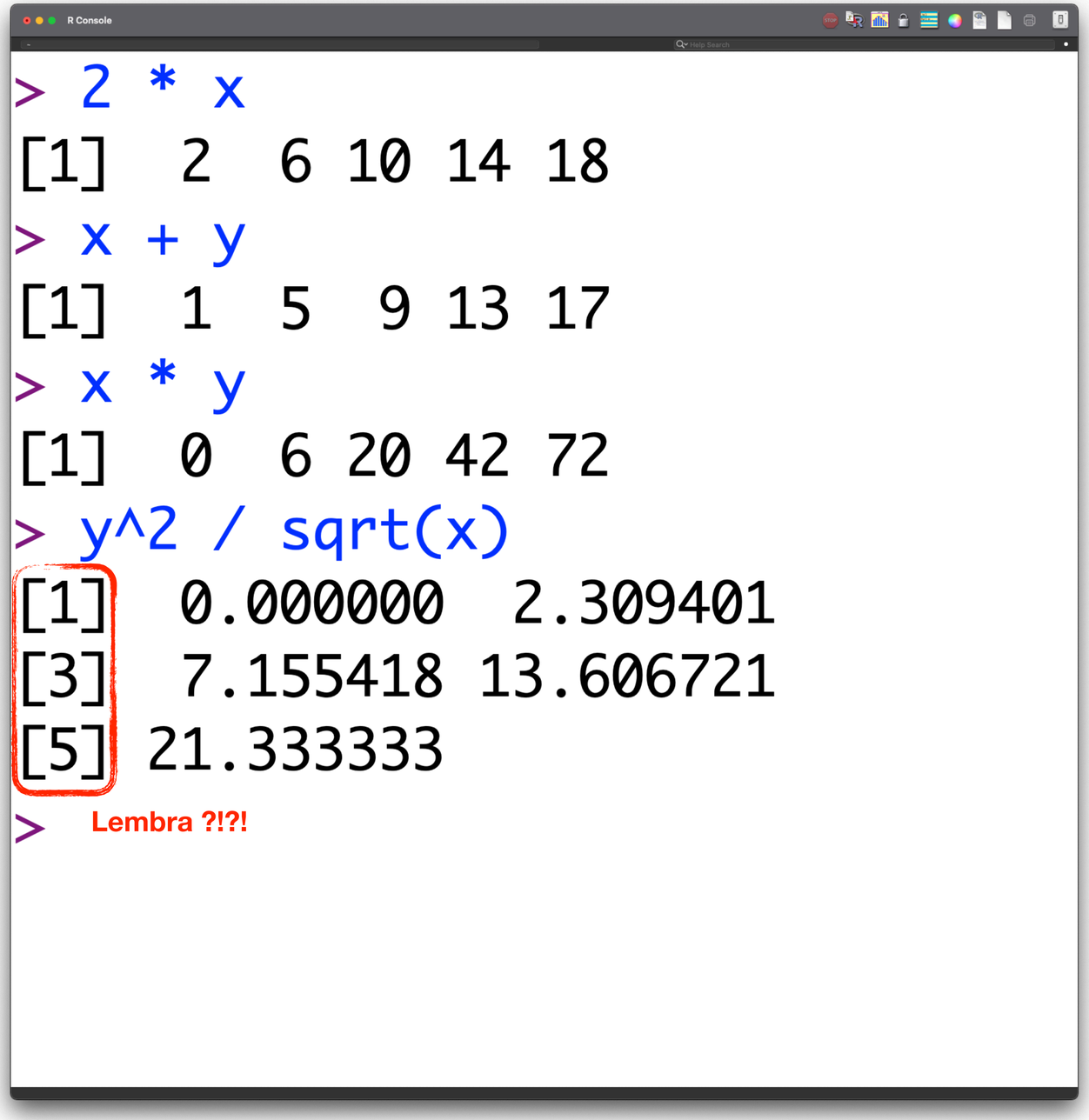
- Podemos salvar vetores em objetos; basta colocar os valores entre c( )
- Com isso, você poderá realizar cálculos vetoriais



R

Uma calculadora elegante

- Podemos salvar vetores em objetos; basta colocar os valores entre c( )
- Com isso, você poderá realizar cálculos vetoriais



R

Uma calculadora elegante

- Podemos salvar vetores em objetos; basta colocar os valores entre c( )
- Com isso, você poderá realizar cálculos vetoriais
- Assim, fica muito fácil criar gráficos com o R

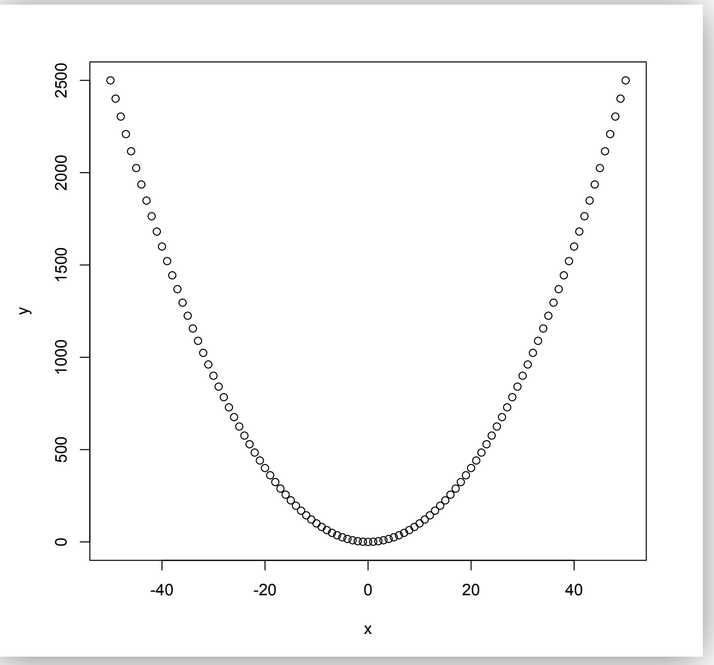
R Console

```
> x = -50:50
> y = x^2
> plot(x,y)
>
```

➡ Cria uma sequência de -50 a 50

➡ Toma o quadrado de cada valor da sequência x

➡ Cria um gráfico de x por y



R

Uma calculadora elegante

- Podemos salvar vetores em objetos; basta colocar os valores entre c( )
- Com isso, você poderá realizar cálculos vetoriais
- Assim, fica muito fácil criar gráficos com o R

R Console

```
> x = -50:50
> y = x^2
> plot(x,y)
> qplot(x,y)
>
```

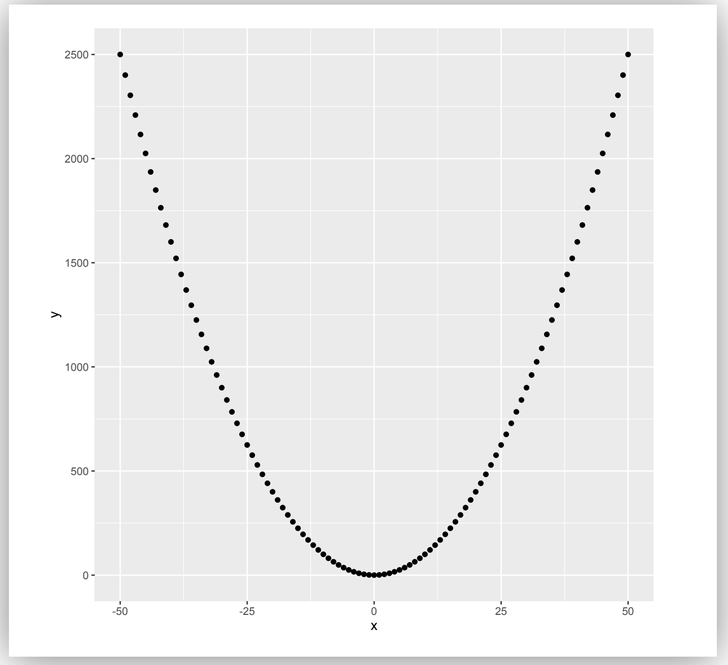
➡ Cria uma sequência de -50 a 50

➡ Toma o quadrado de cada valor da sequência x

➡ Cria um gráfico de x por y



Ainda falaremos sobre o qplot !!



SUA VEZ

QUAL O RESULTADO DO COMANDO ABAIXO?

```
factorial( round(2.1016) + 1 )
```

SUA VEZ

QUAL O RESULTADO DO COMANDO ABAIXO?

```
factorial( round(2.1016) + 1 )
```

SUA VEZ

QUAL O RESULTADO DO COMANDO ABAIXO?

```
factorial( round(2.1016) + 1 )
```



```
factorial( 2 + 1 )
```



```
factorial( 3 )
```

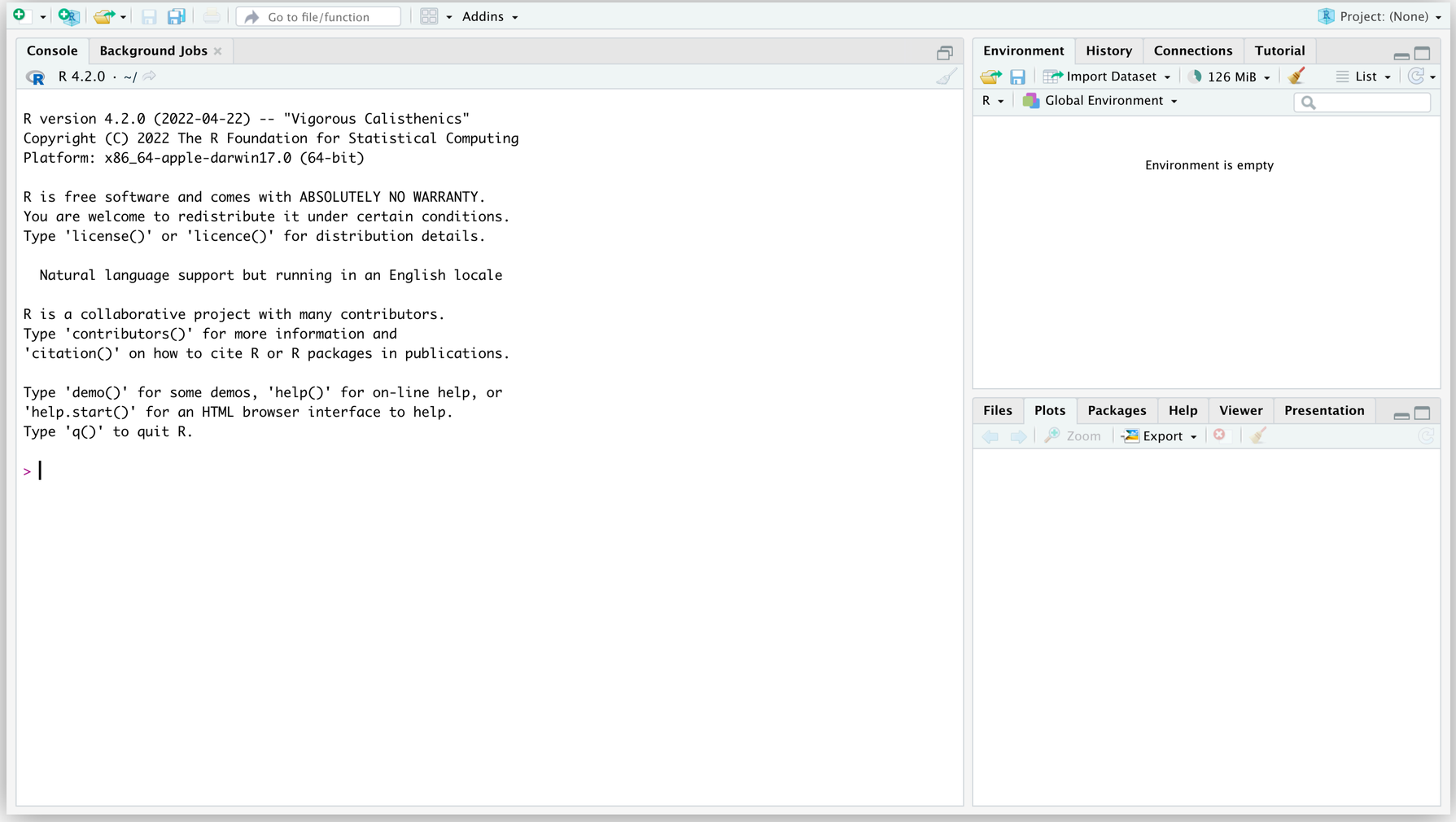


```
6
```

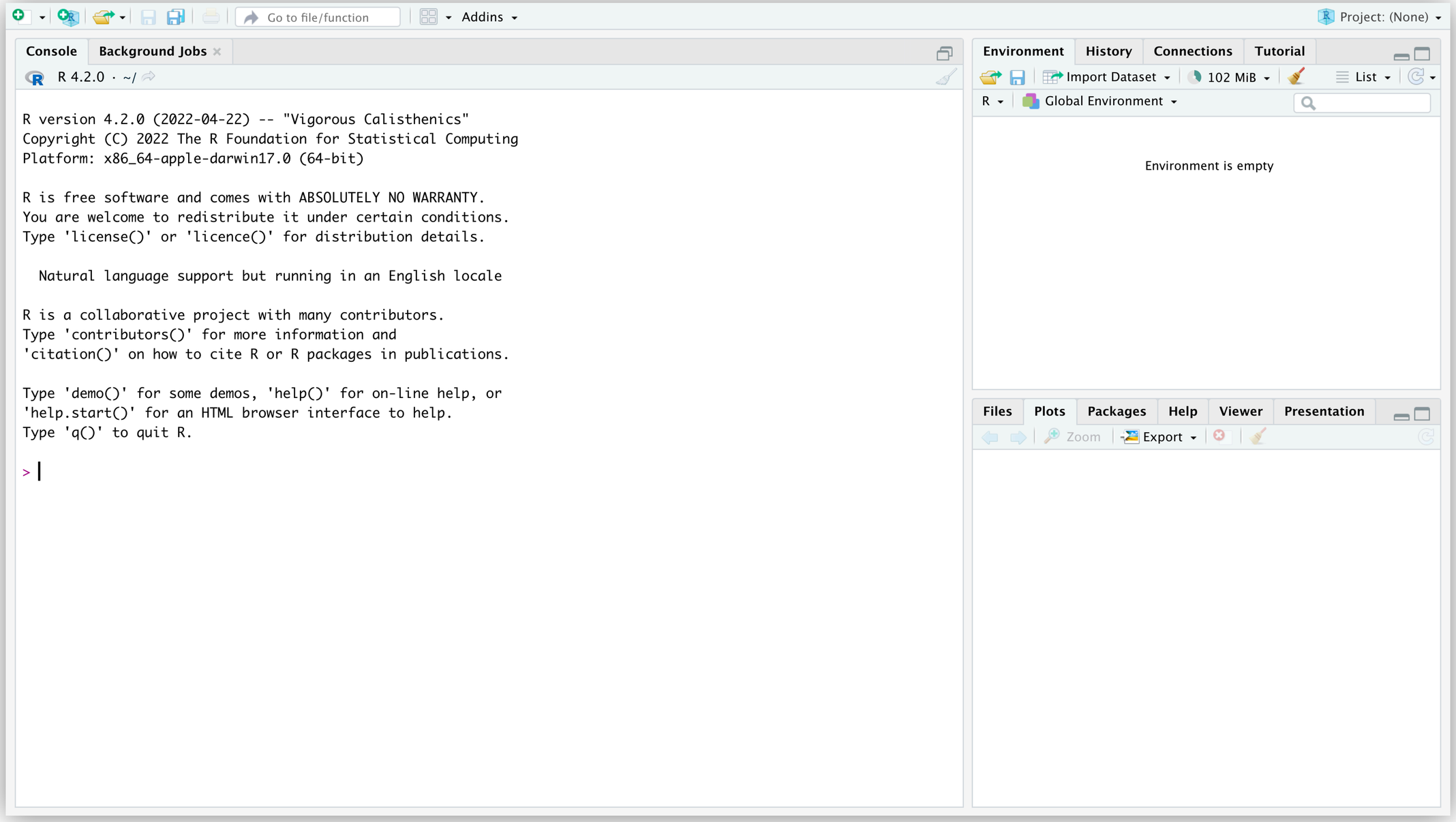

RStudio®

RStudio

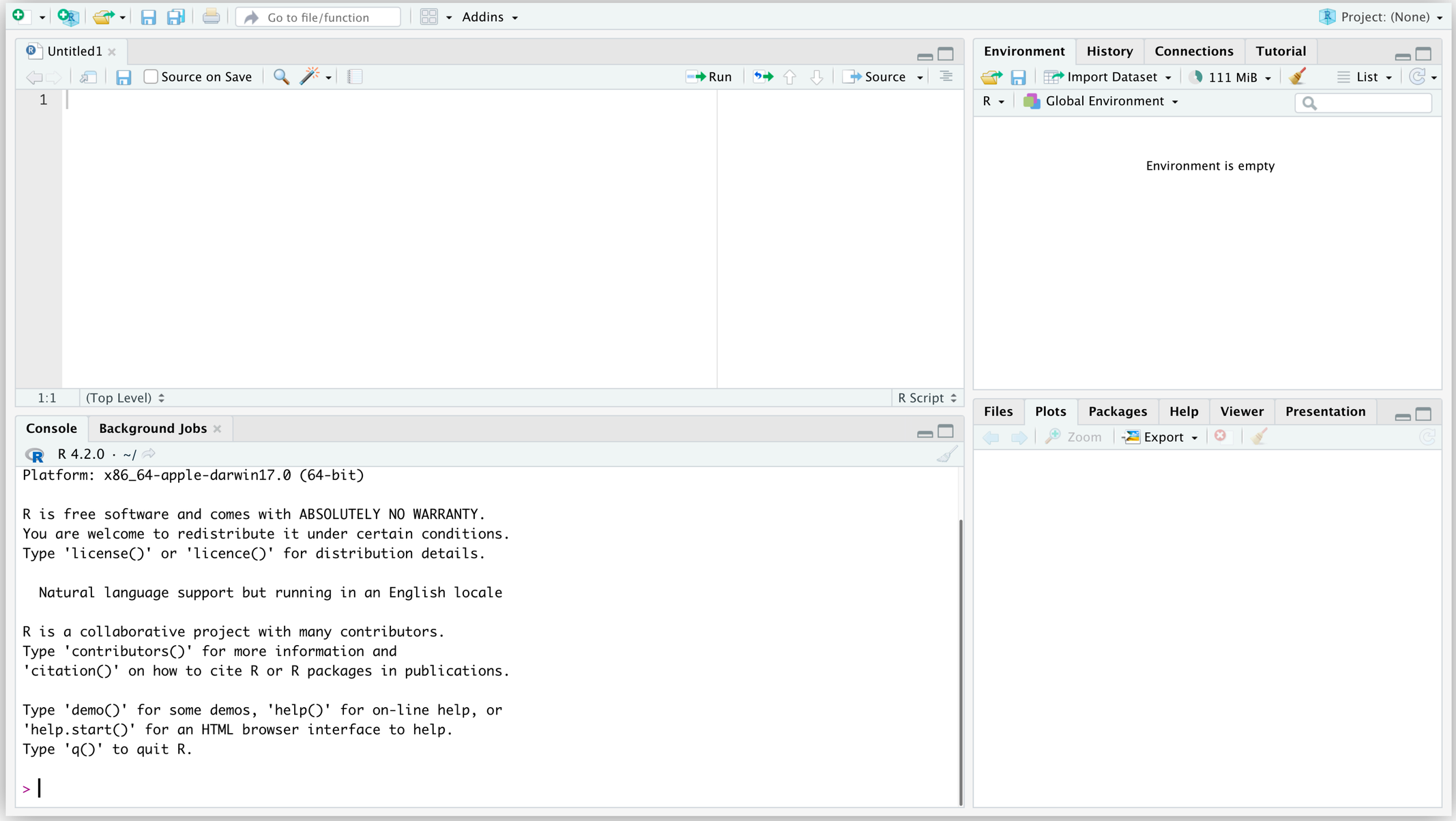
Um ambiente integrado de trabalho



A janela do RStudio é dividida em 3-4 painéis.



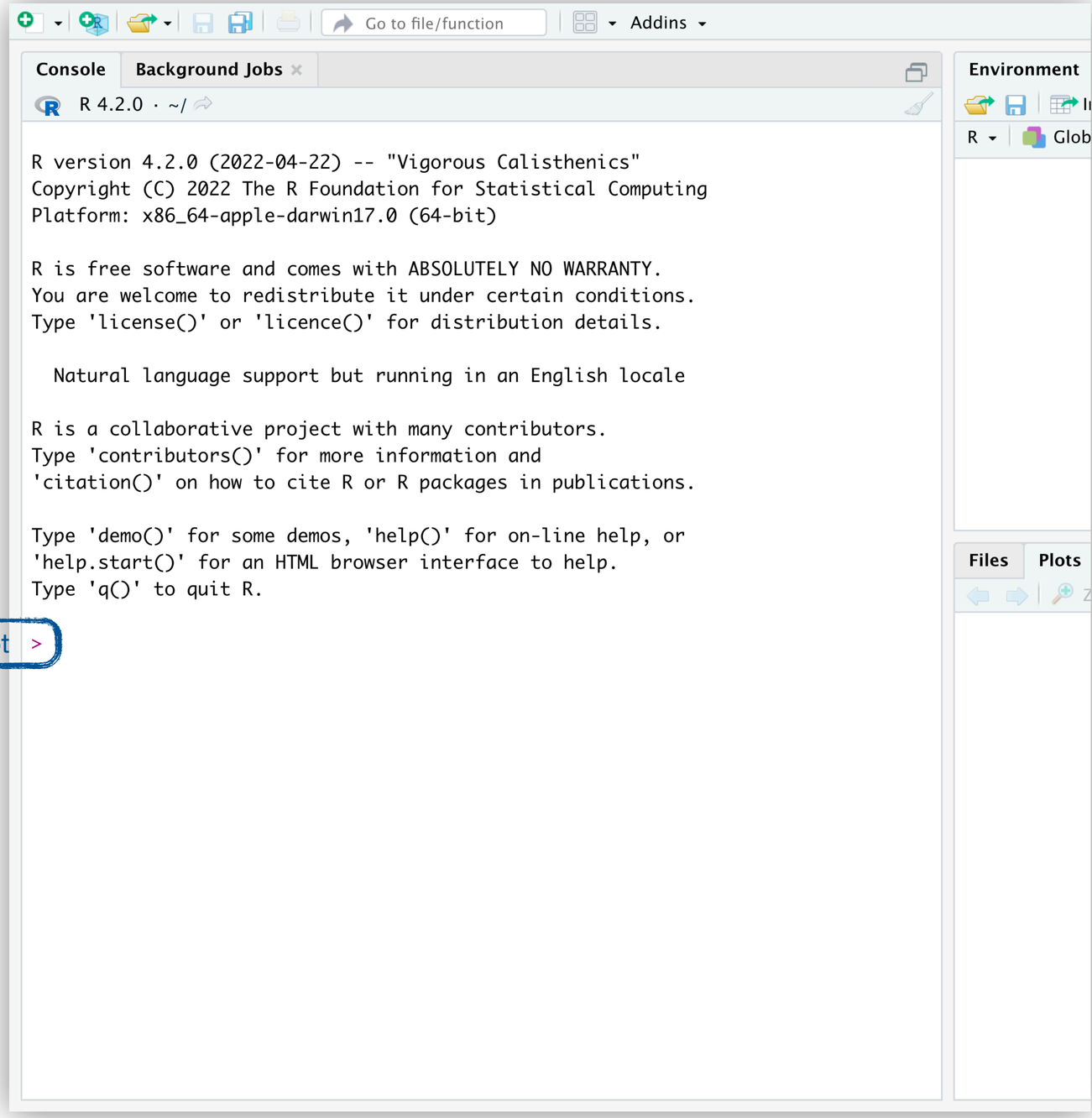
A janela do RStudio é dividida em 3-4 painéis.



R Console

Expandido

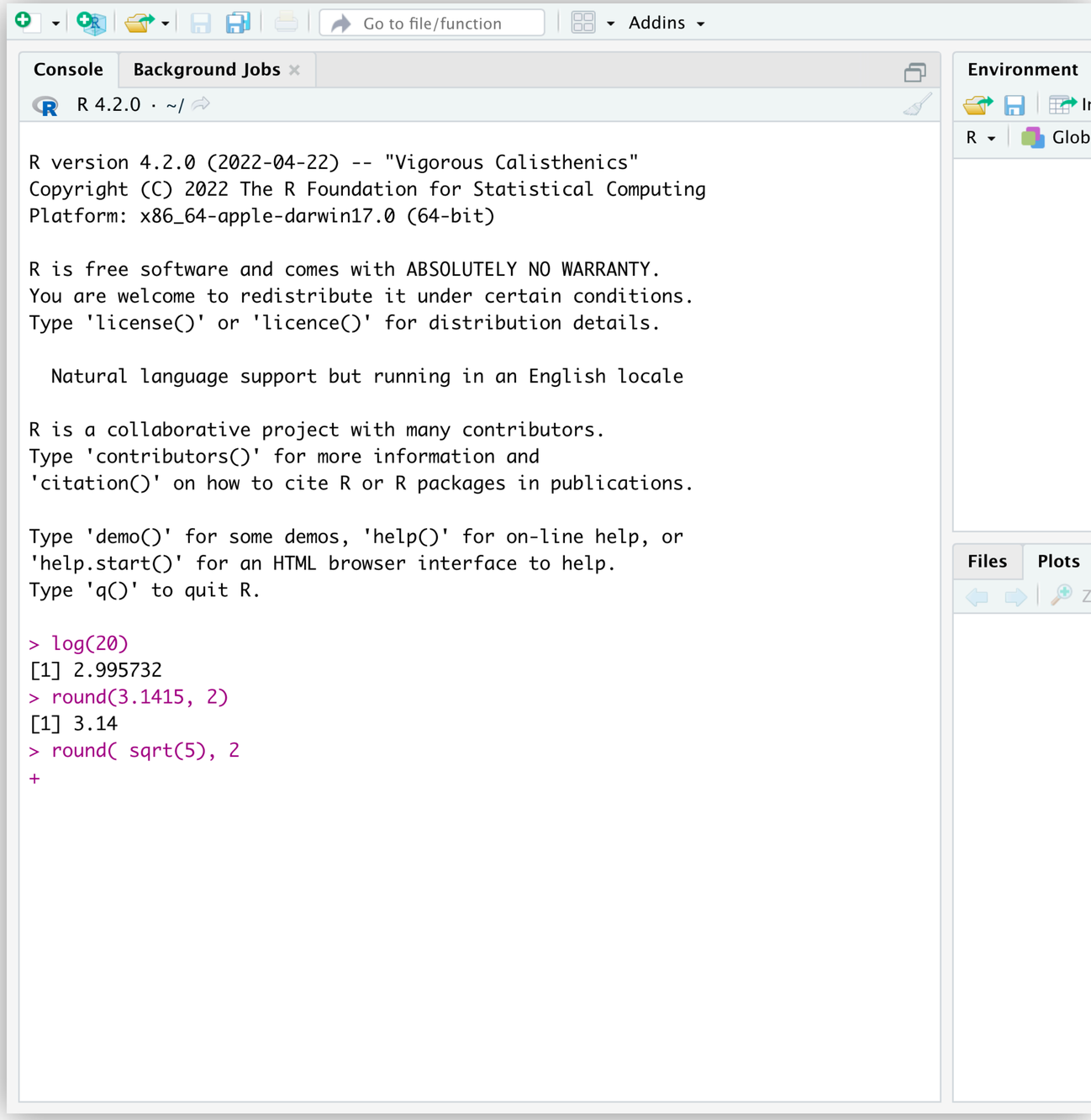
- O console nos oferece um local para executar os comandos escritos na linguagem R
- Na visualização com 3 painéis, ele ocupa todo o lado esquerdo do RStudio



R Console

Expandido

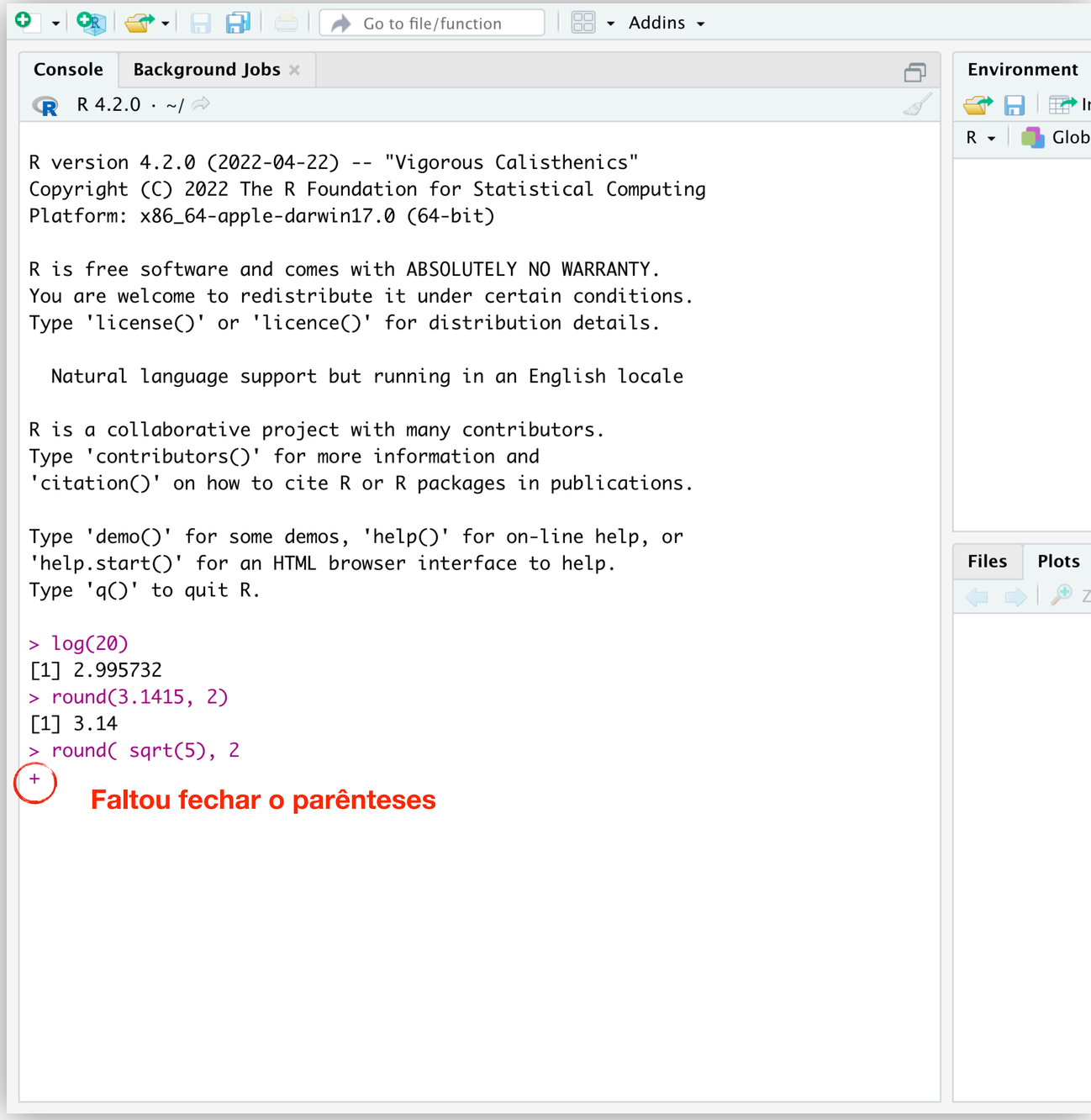
- O console nos oferece um local para executar os comandos escritos na linguagem R
- Na visualização com 3 painéis, ele ocupa todo o lado esquerdo do RStudio
- Se um “+” aparecer no prompt, provavelmente você não terminou seu código



R Console

Expandido

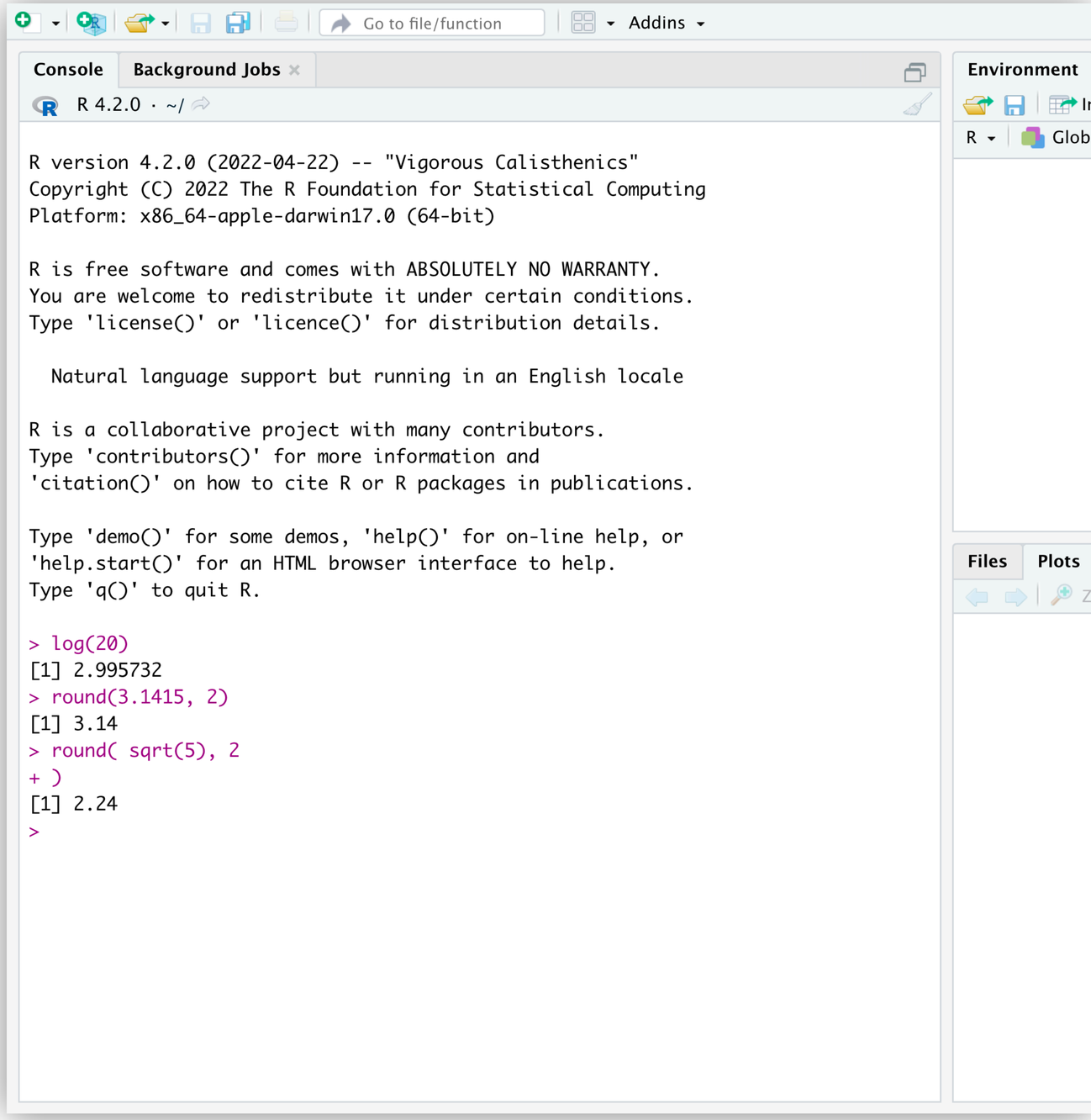
- O console nos oferece um local para executar os comandos escritos na linguagem R
- Na visualização com 3 painéis, ele ocupa todo o lado esquerdo do RStudio
- Se um “+” aparecer no prompt, provavelmente você não terminou seu código



R Console

Expandido

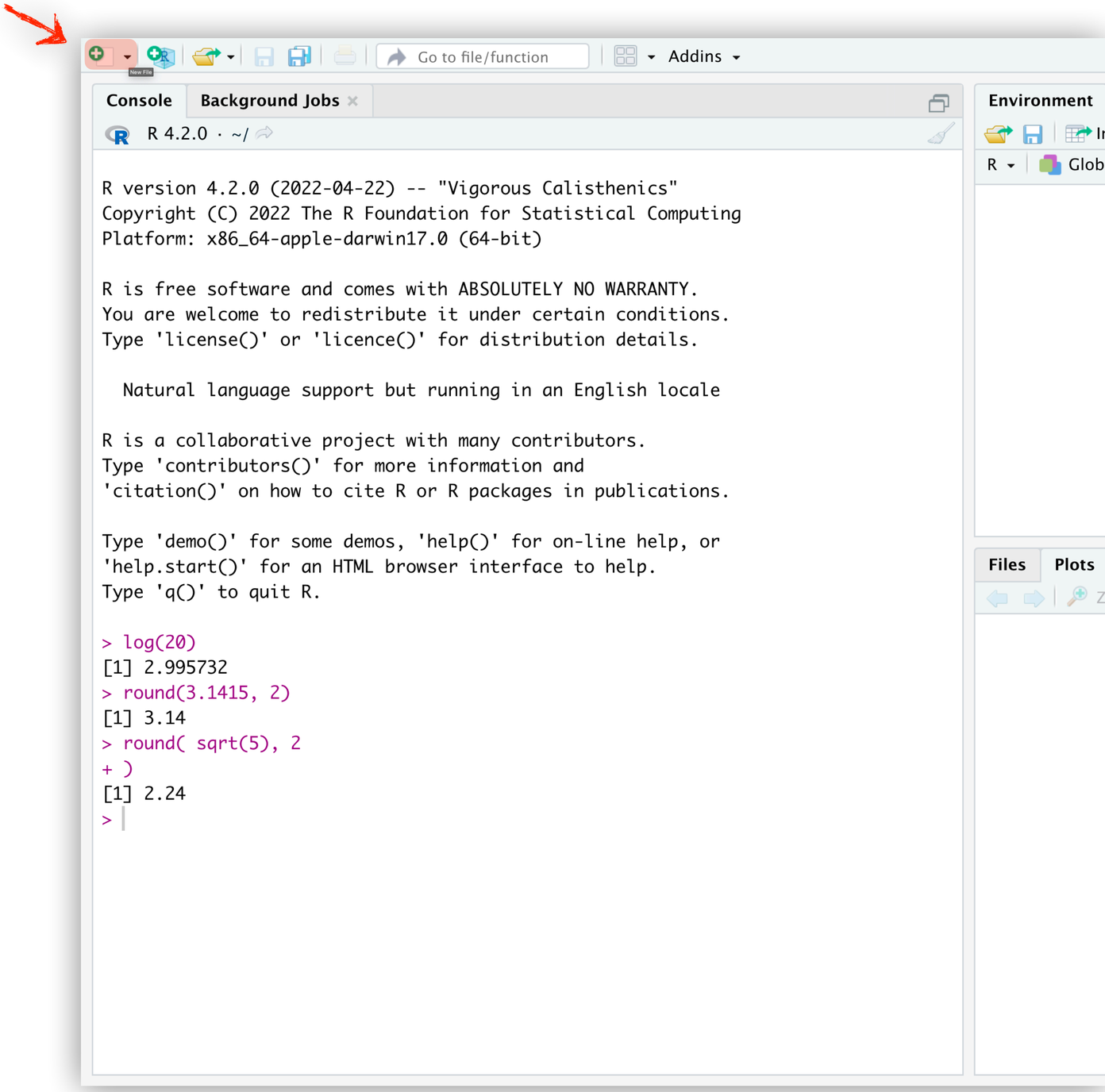
- O console nos oferece um local para executar os comandos escritos na linguagem R
- Na visualização com 3 painéis, ele ocupa todo o lado esquerdo do RStudio
- Se um “+” aparecer no prompt, provavelmente você não terminou seu código



R Script

Compartilhado

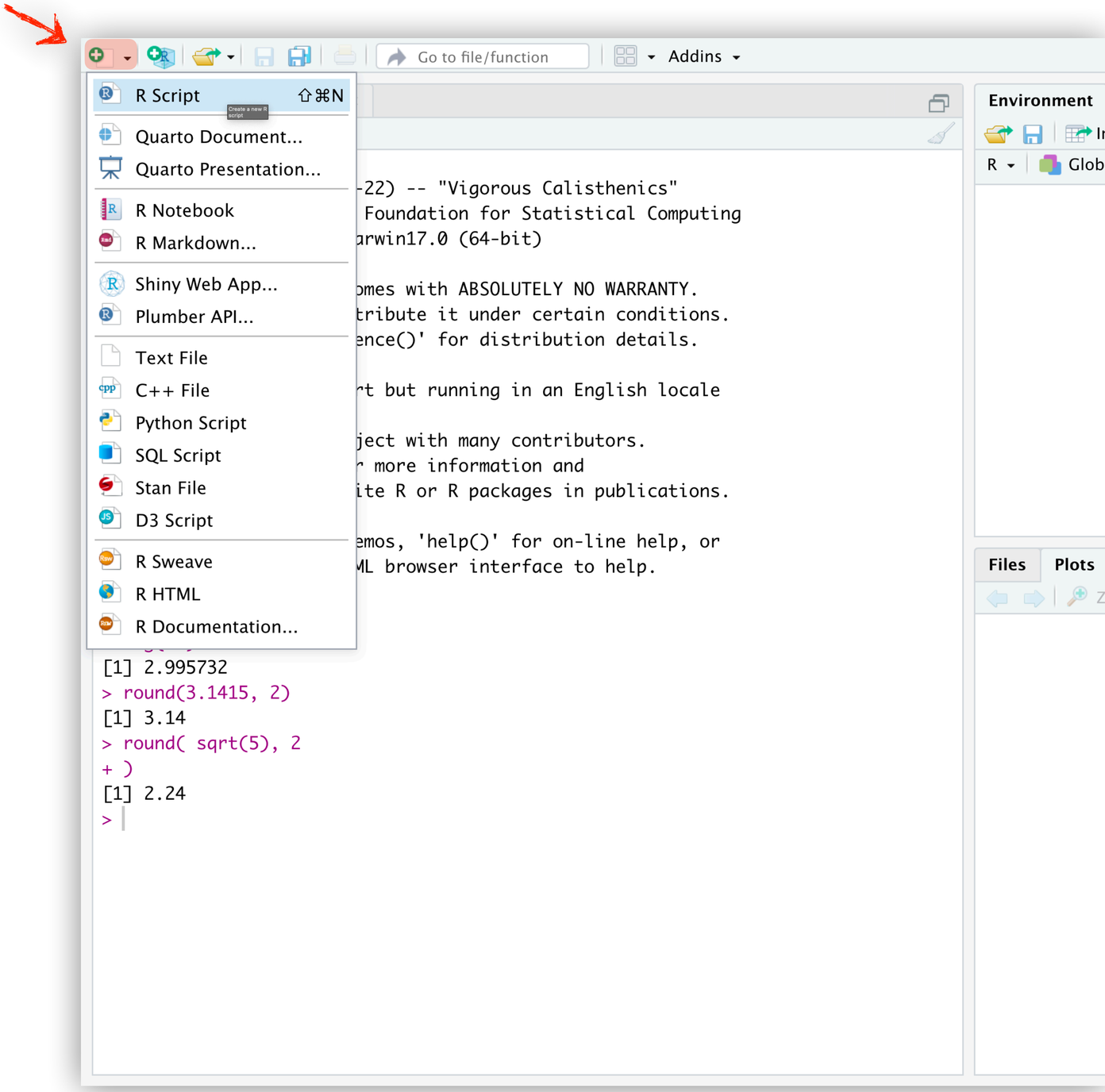
- Ao adicionarmos um *script* visualizaremos 4 painéis



R Script

Compartilhado

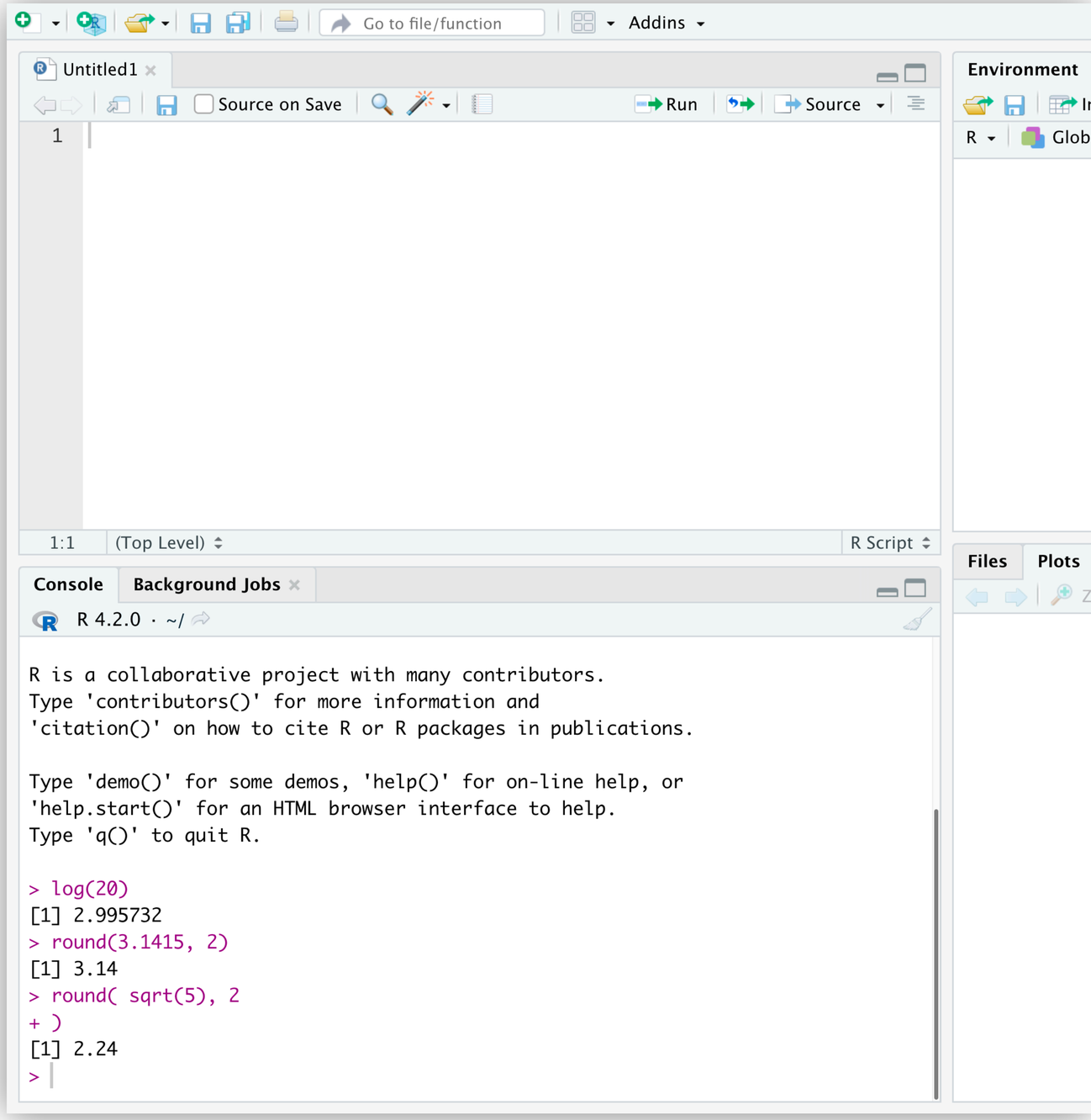
- Ao adicionarmos um *script* visualizaremos 4 painéis



R Script

Compartilhado

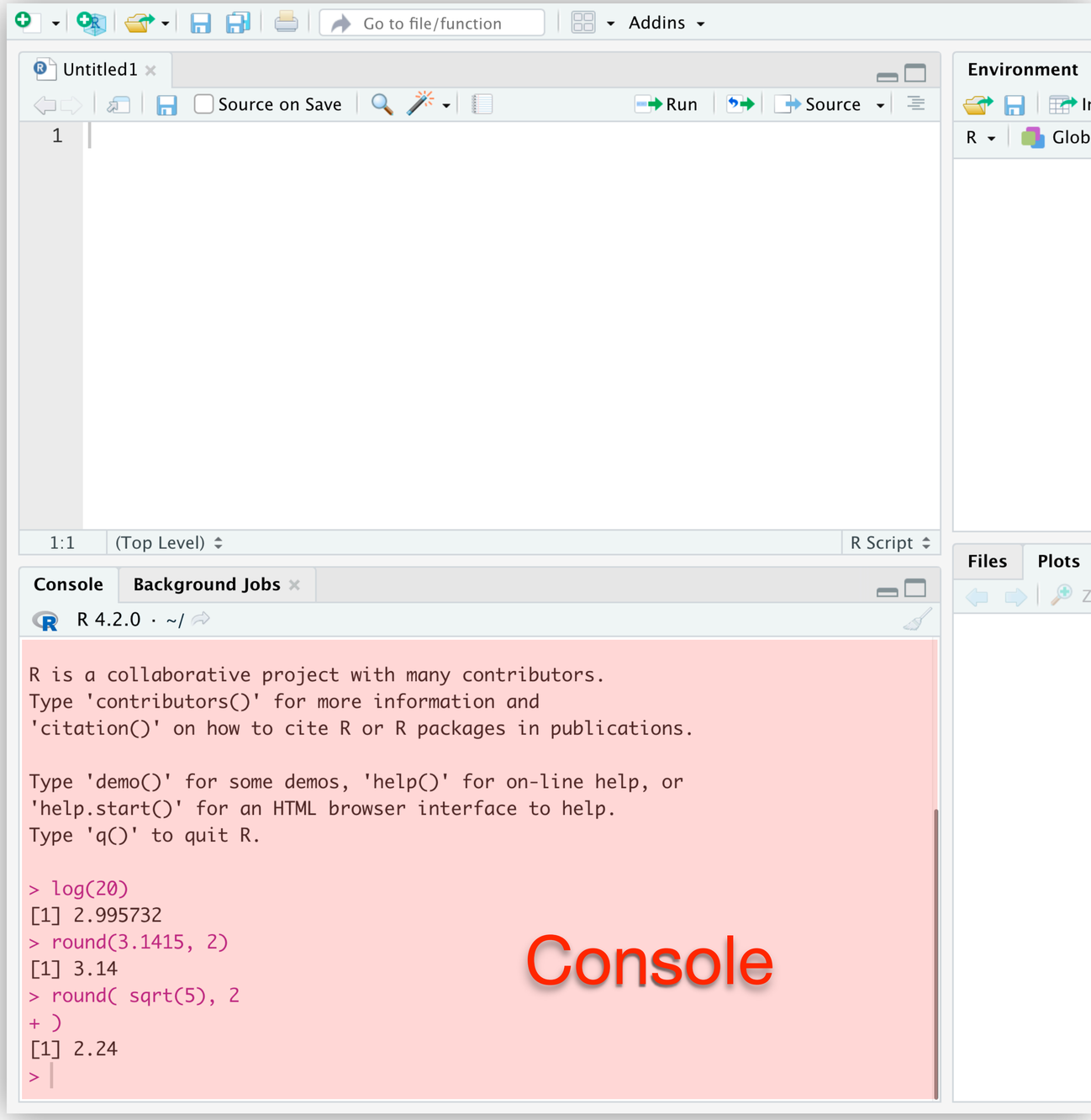
- Ao adicionarmos um *script* visualizaremos 4 painéis



R Script

Compartilhado

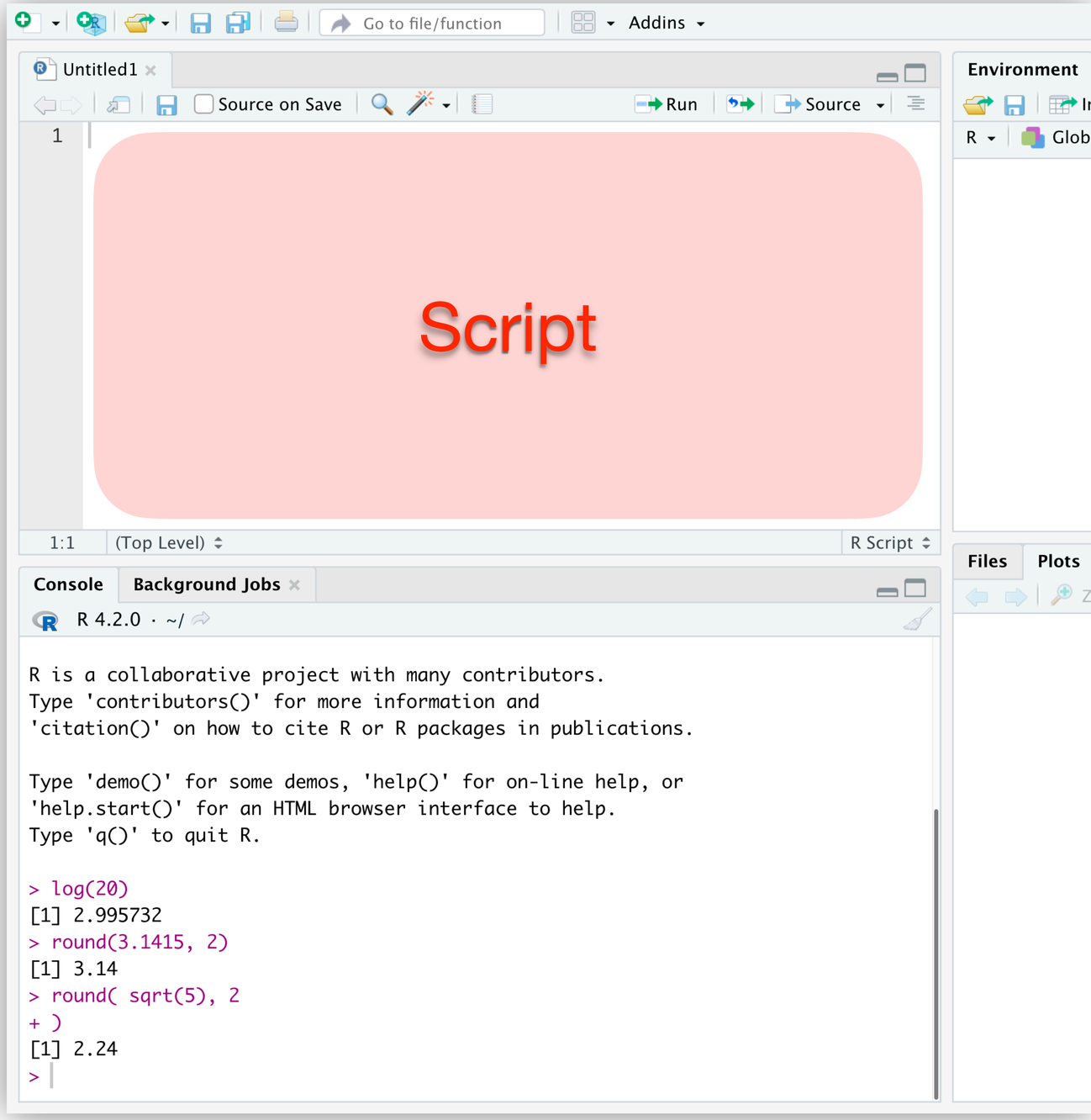
- Ao adicionarmos um *script* visualizaremos 4 painéis



R Script

Compartilhado

- Ao adicionarmos um *script* visualizaremos 4 painéis

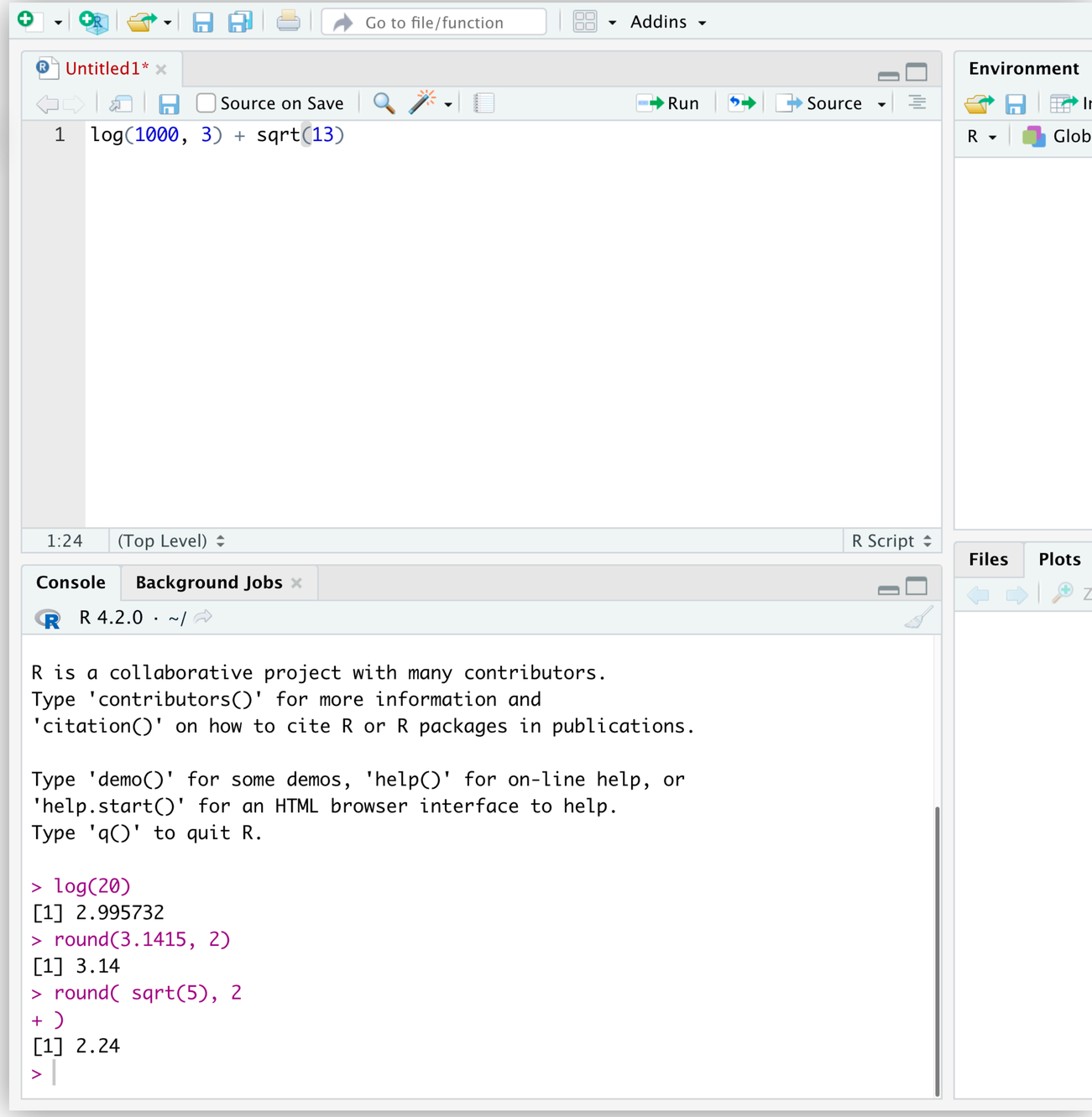


R Script

Compartilhado

- Ao adicionarmos um *script* visualizaremos 4 painéis

    1. Escreva seu código no script



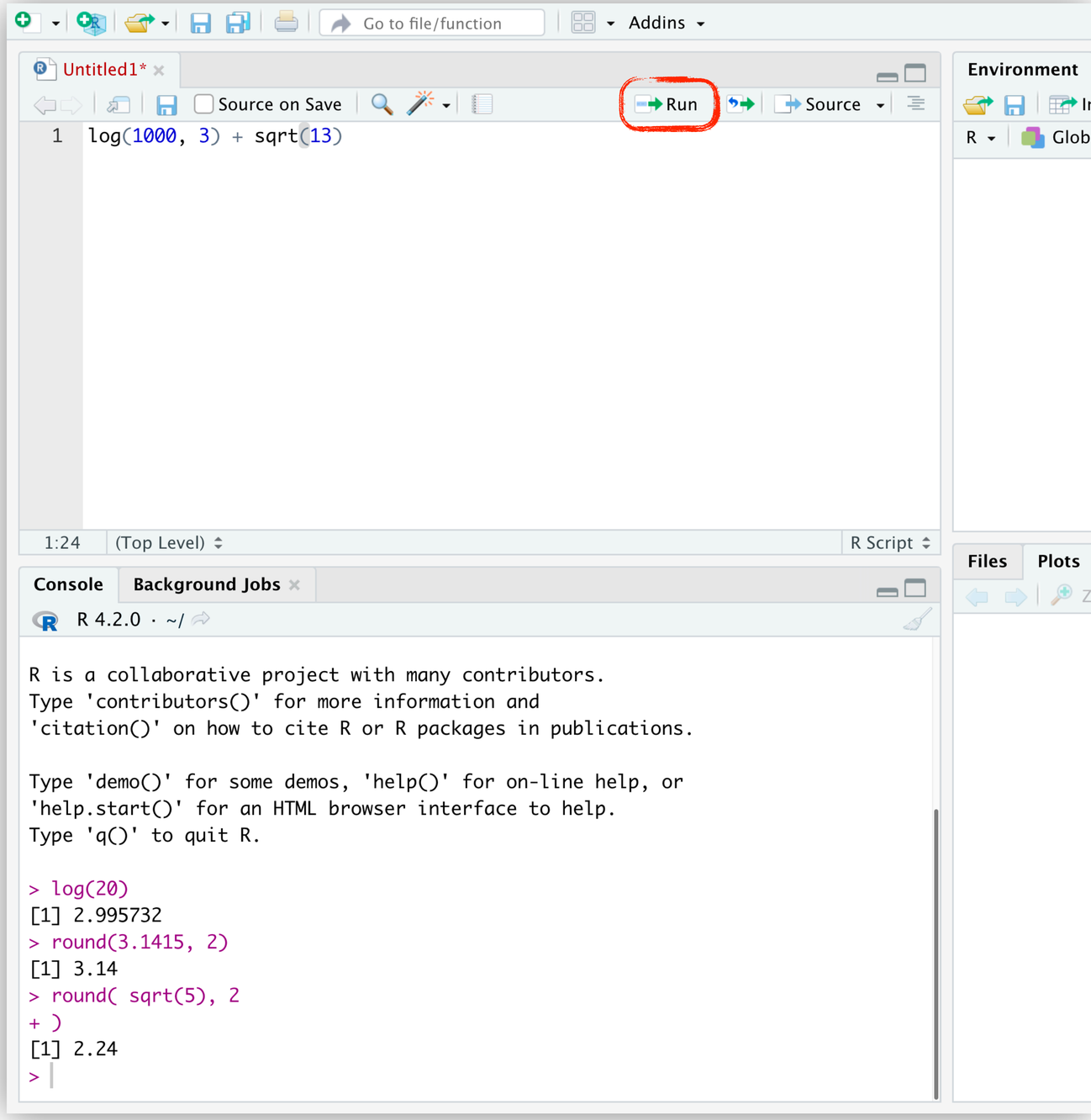
R Script

Compartilhado

- Ao adicionarmos um *script* visualizaremos 4 painéis

    1. Escreva seu código no script

    2. Execute o comando com o botão *Run*

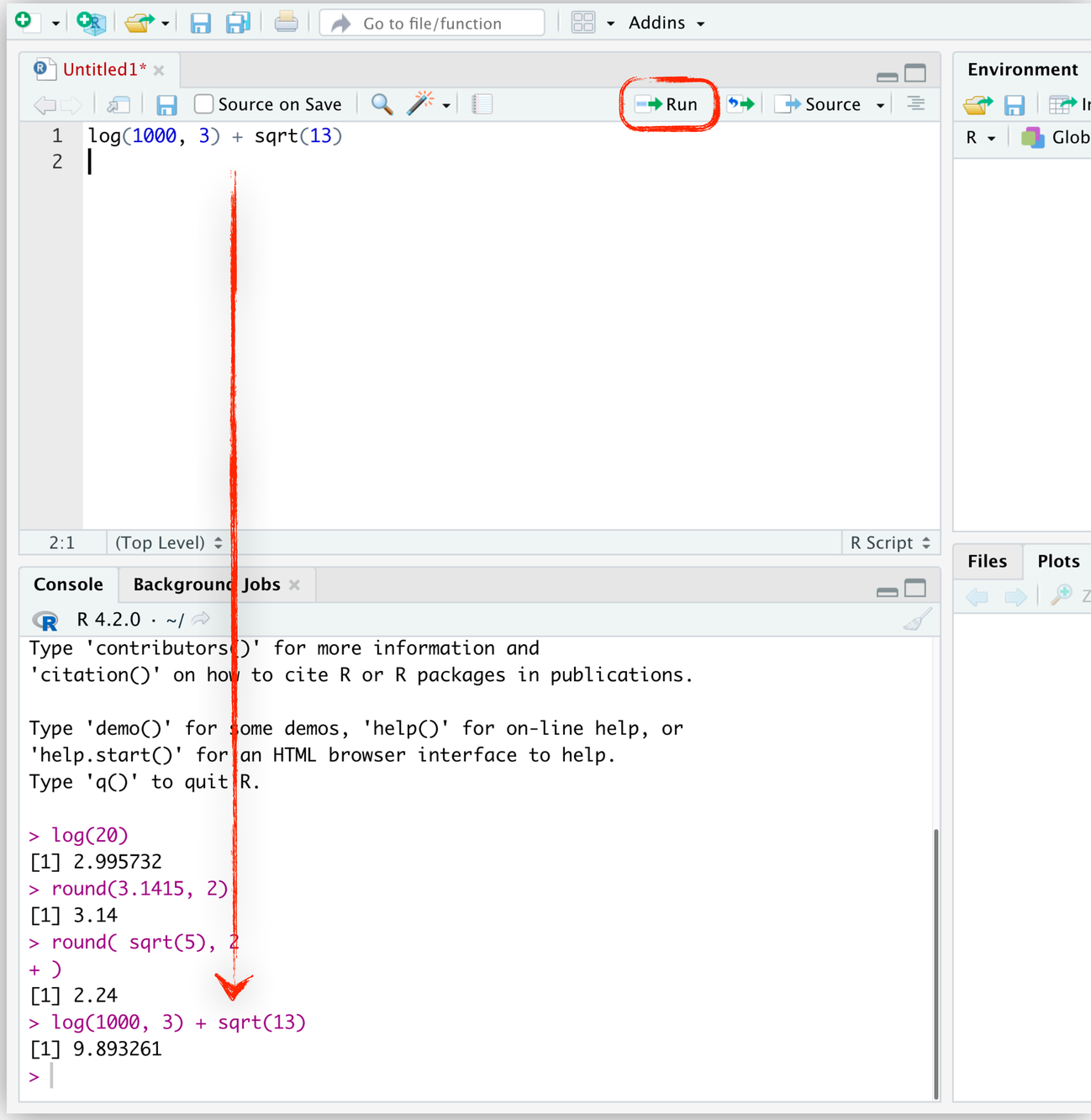


R Script

Compartilhado

- Ao adicionarmos um *script* visualizaremos 4 painéis

    1. Escreva seu código no script
    2. Execute o comando com o botão *Run*

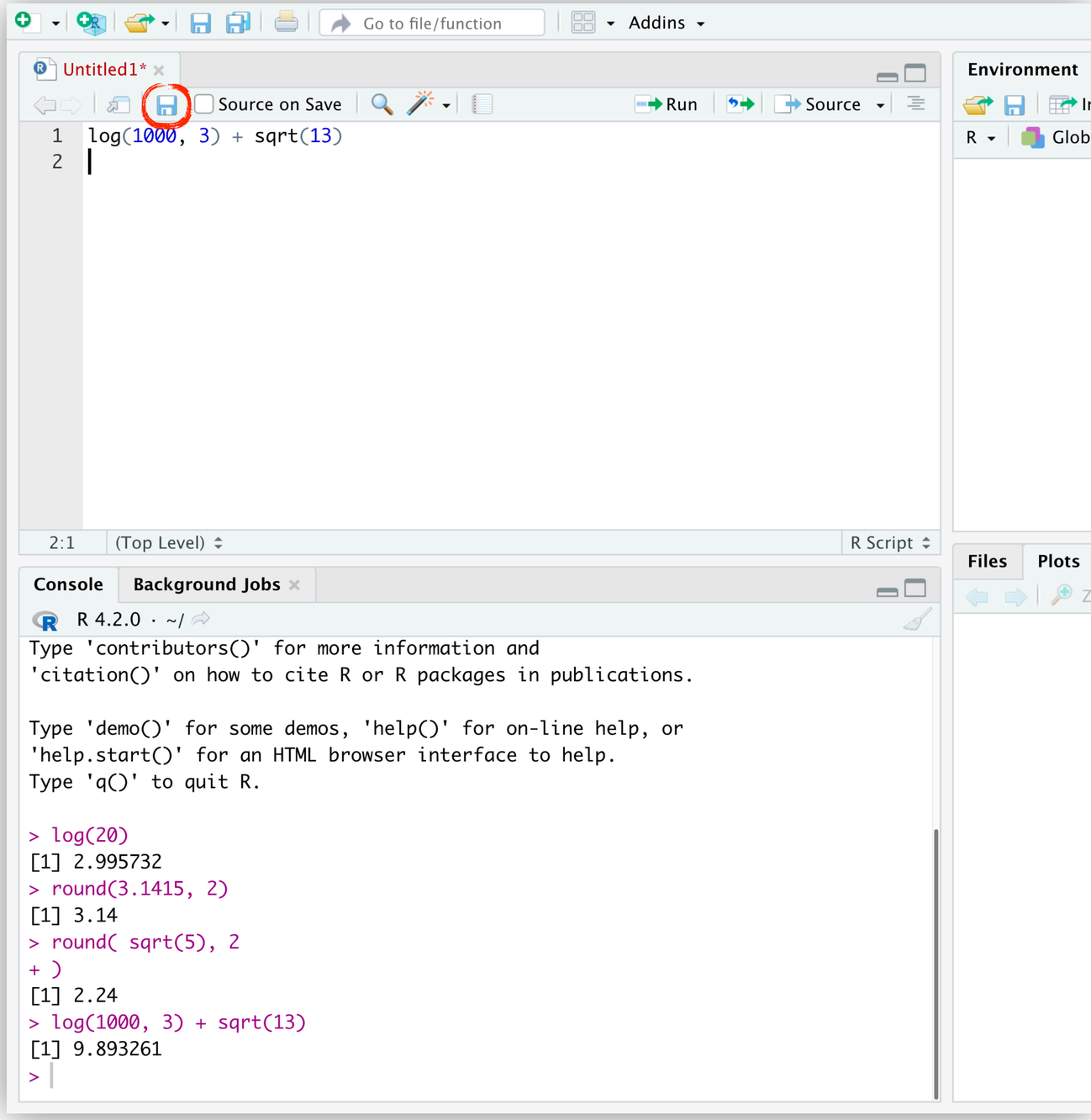


R Script

Compartilhado

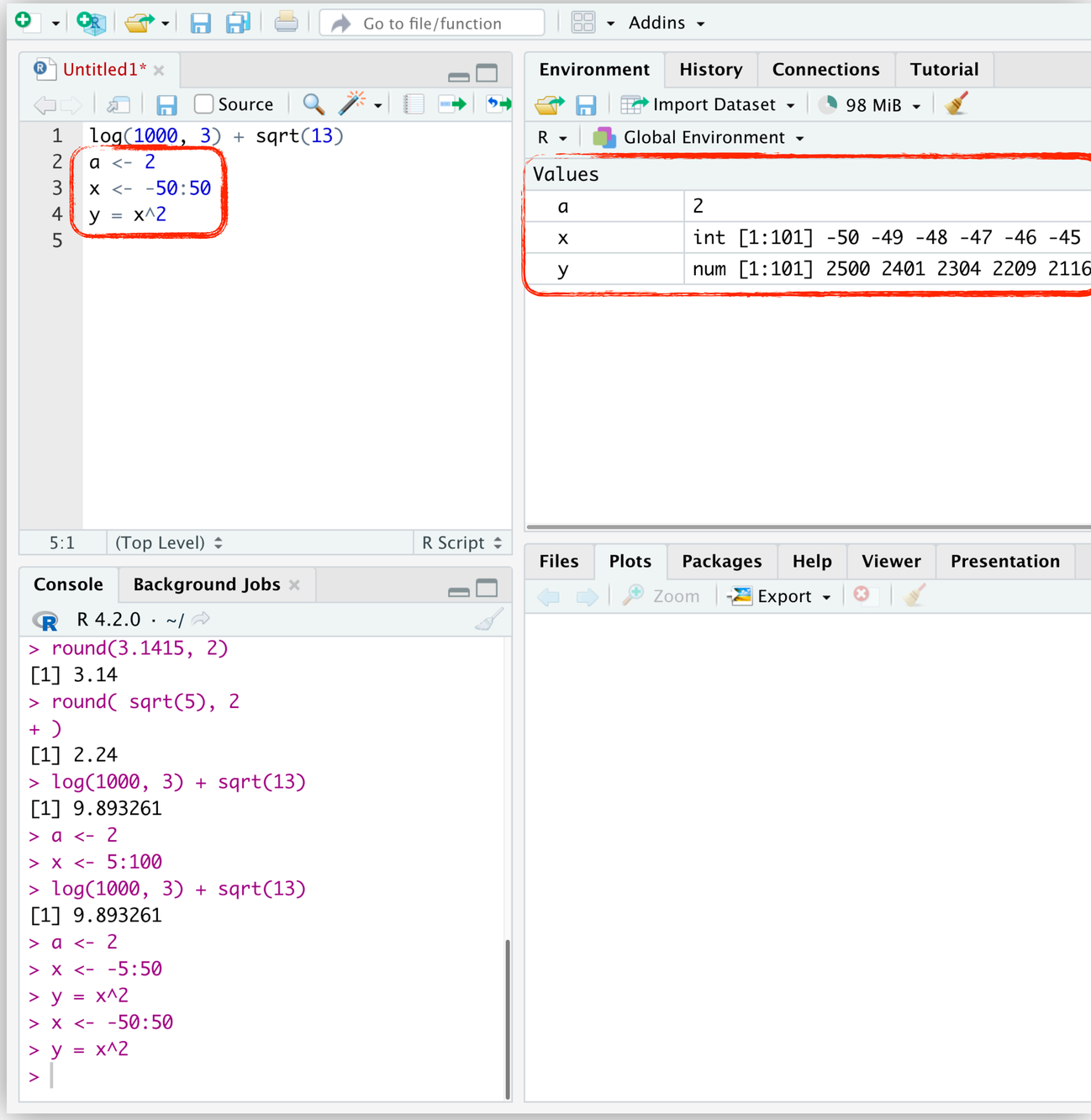
- Ao adicionarmos um *script* visualizaremos 4 painéis

    1. Escreva seu código no script
    2. Execute o comando com o botão *Run*
    3. Salve o seu script



Environment and History

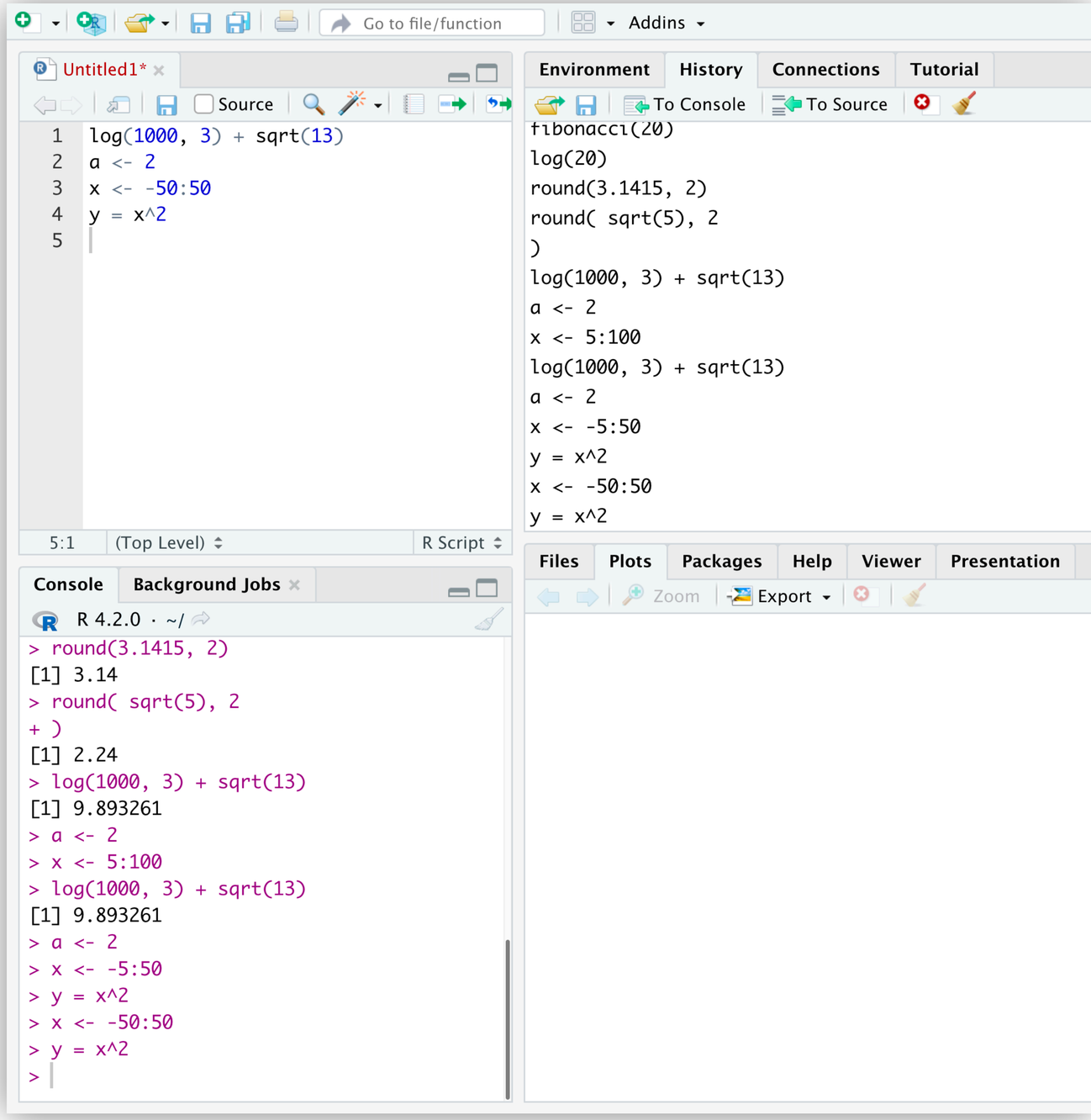
- Todo objeto criado no RStudio aparece na aba *Environment* que fica à direita do script



Environment and History

- Todo objeto criado no RStudio aparece na aba *Environment* que fica à direita do script
- Ao lado você irá encontrar a aba *History* que contém um histórico dos comandos executados.

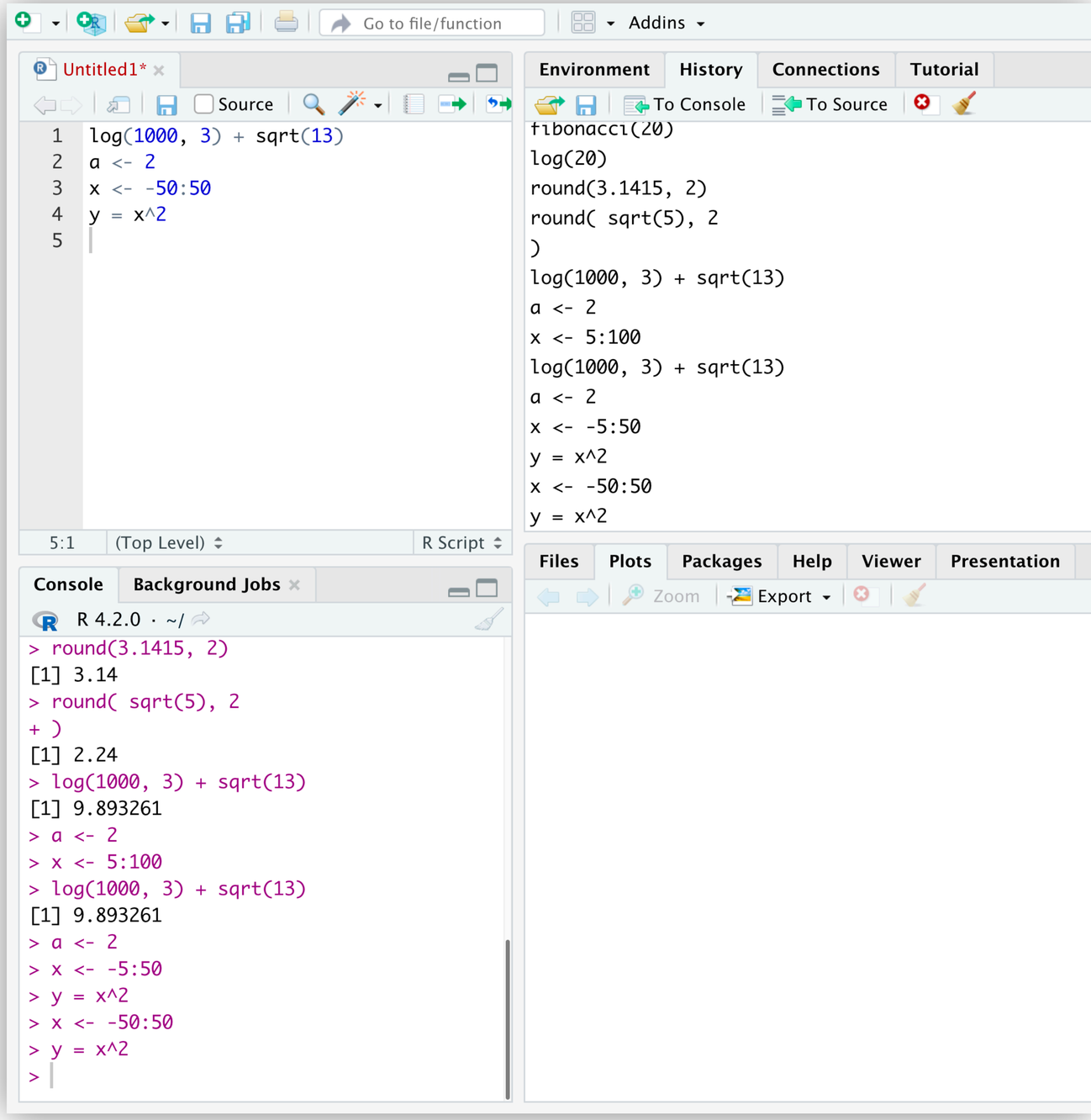
✓ Serão registrados tanto os dados executados no script como no console



Gráficos

E outros

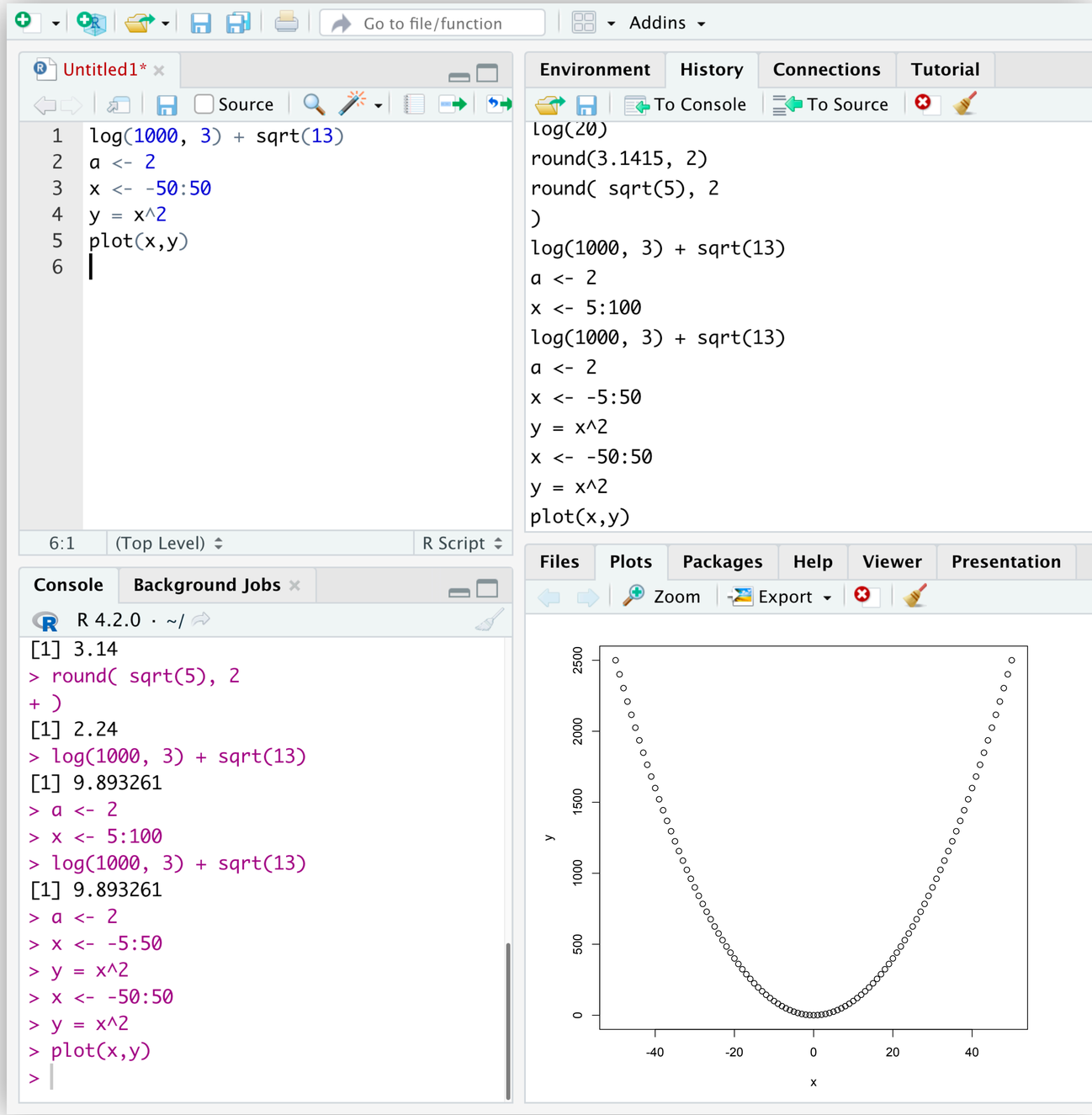
- Logo abaixo você verá uma sequência com 6 abas



Gráficos

E outros

- Logo abaixo você verá uma sequência com 6 abas
- Na aba *Plots* você poderá ver uma pré-visualização dos gráficos gerados pelos seus comandos
- Mais à frente iremos aprender sobre as outras abas



Objetos no R

Um pouco mais

Como já adiantamos, você pode atribuir objetos ao R por meio do sinal de menor seguido do sinal de menos: <-

Alternativamente, você pode utilizar o sinal de igual: =

```
foo <- 42
```

Como já adiantamos, você pode atribuir objetos ao R por meio do sinal de menor seguido do sinal de menos: <-

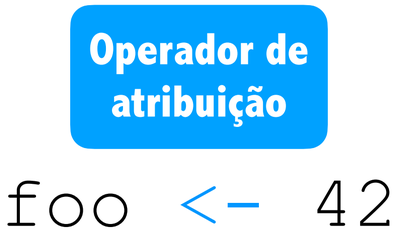
Alternativamente, você pode utilizar sinal de igual: =

Nome do novo objeto

```
foo <- 42
```

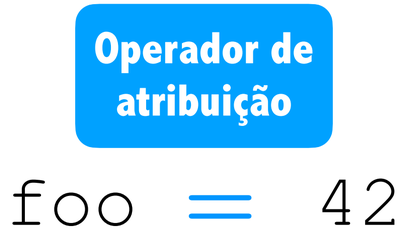
Como já adiantamos, você pode atribuir objetos ao R por meio do sinal de menor seguido do sinal de menos: <-

Alternativamente, você pode utilizar sinal de igual: =



Como já adiantamos, você pode atribuir objetos ao R por meio do sinal de menor seguido do sinal de menos: <-

Alternativamente, você pode utilizar sinal de igual: =



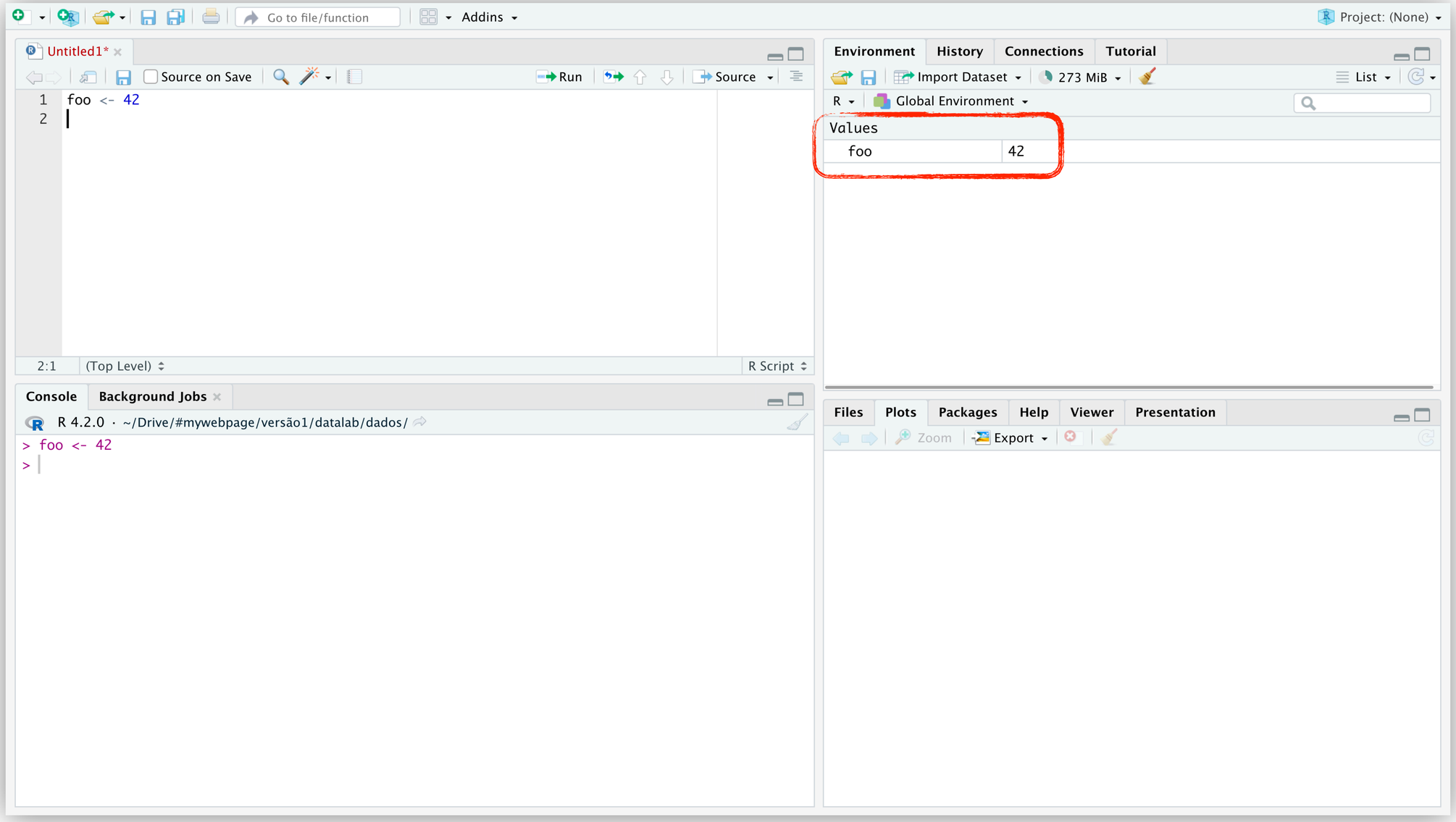
Como já adiantamos, você pode atribuir objetos ao R por meio do sinal de menor seguido do sinal de menos: <-

Alternativamente, você pode utilizar sinal de igual: =

Informação a ser guardada no objeto

```
foo <- 42
```

Quando você cria um objeto no R, ele aparece na aba Environment



Fluxo comum no R

- Salve a saída de uma função em um objeto para utilizar em um segunda função

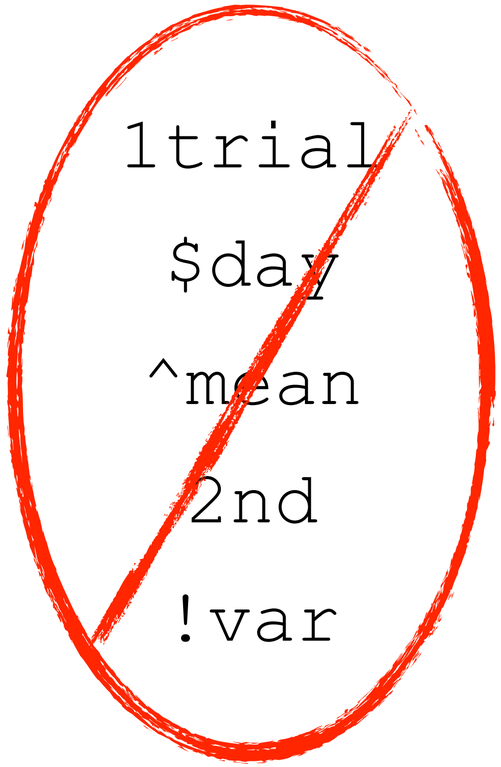
```
foo <- round(3.1415) + 1
foo
[1] 4


factorial(foo)
[1] 24
```

Nomes de objetos

- Não pode começar com números ou símbolos que já possuem uma finalidade
- É sábio evitar nomes já em uso

```
a
B
F00
my_var
.day
```



Minúsculo vs Maiúsculo

- O R trata minúsculo e maiúsculo como elementos distintos
- Como exemplo, os objetos abaixo serão diferentes

`a`	`A`
`b`	`B`
`var`	`VAR`
`my_VAR`	`MY_var`

rm

- Você poderá remover um objeto por meio do comando `rm`

```
foo
[1] 4

rm(foo)
foo
[1] Error: object 'foo' not found
```

rm

- Pode ser útil se você sobrescrever um objeto já pré-definido pelo R

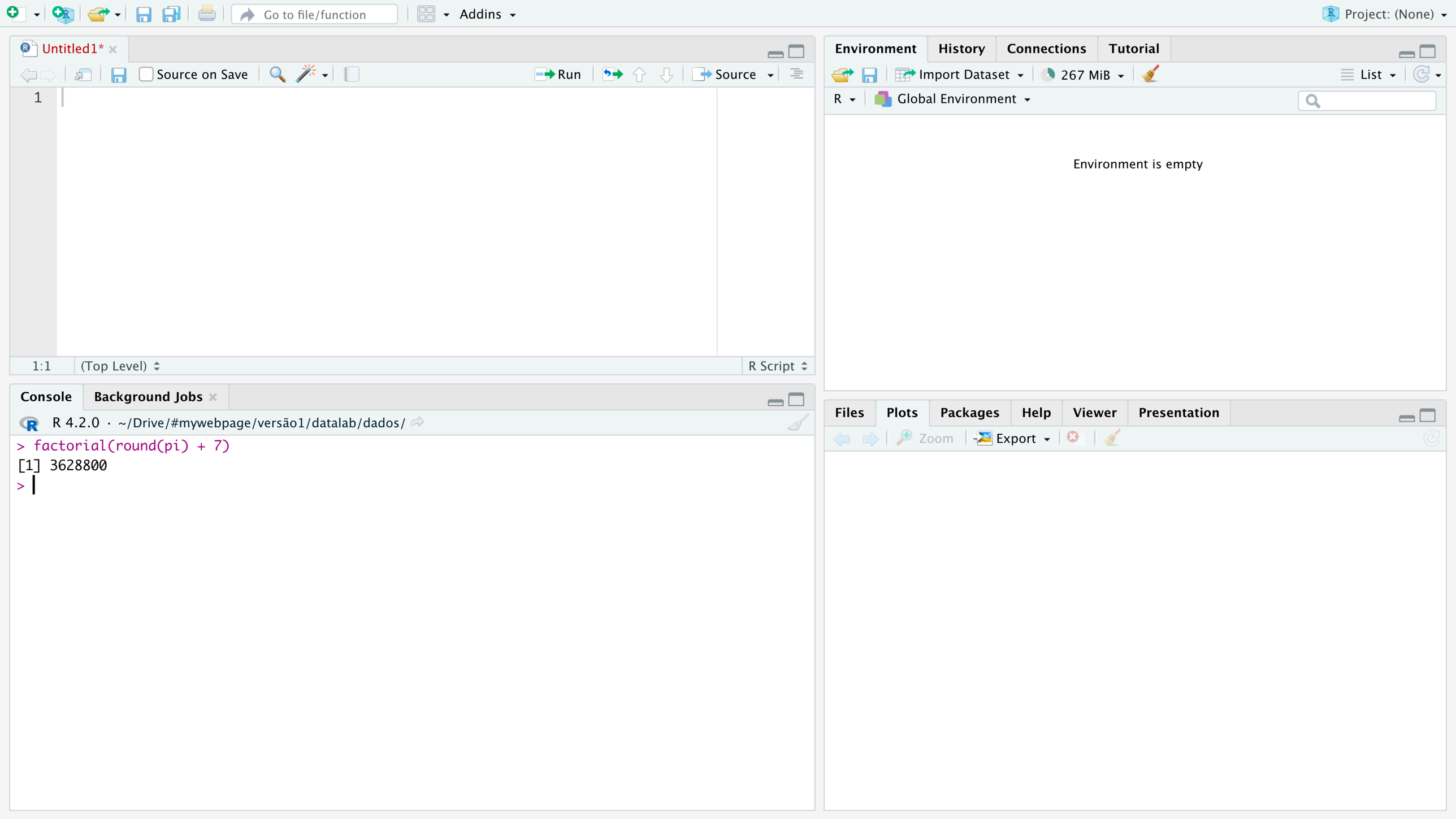
```
pi
[1] 3.141593

pi <- 1
pi
[1] 1

rm(pi)
pi
[1] 3.141593
```

Atalhos

Facilitando


Go to file/function
Addins
Project: (None)
Untitled1*
Source on Save
Run
Source
1
1:1
(Top Level)
R Script
Environment
History
Connections
Tutorial
Import Dataset
267 MiB
List
R
Global Environment
Environment is empty
Console
Background Jobs
R 4.2.0 · ~/Drive/#mywebpage/versão1/datalab/dados/
> factorial(round(pi) + 7)
[1] 3628800
>
Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export

Go to file/function

Addins

Project: (None)

Untitled1*

Source on Save

Run

Source

1

1:1

(Top Level)

R Script

Console

Background Jobs

R 4.2.0 · ~/Drive/#mywebpage/versão1/datalab/dados/

```
> factorial(round(pi) + 7)
[1] 3628800
> factorial(round(pi) + 7)
```



Environment

History

Connections

Tutorial

Import Dataset

267 MiB

List

R

Global Environment

Environment is empty

Files

Plots

Packages

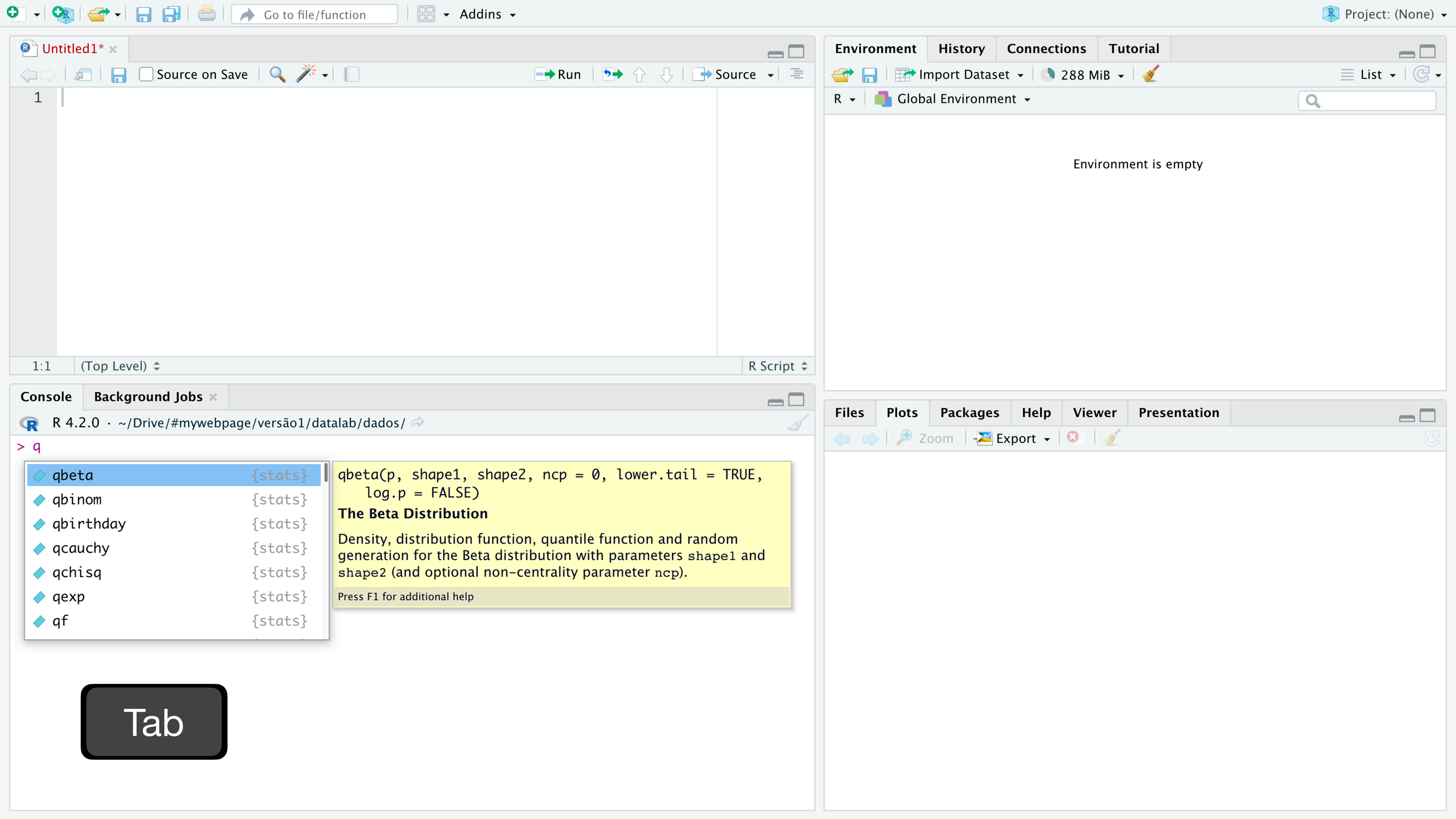
Help

Viewer

Presentation

Zoom

Export


Go to file/function
Addins
Project: (None)
Untitled1*
Source on Save
Run
Source
1
1:1
(Top Level)
R Script
Environment
History
Connections
Tutorial
Import Dataset
288 MiB
List
R
Global Environment
Environment is empty
Console
Background Jobs
R 4.2.0 · ~/Drive/#mywebpage/versão1/datalab/dados/
> q
qbeta {stats}
qbinom {stats}
qbirthday {stats}
qcauchy {stats}
qchisq {stats}
qexp {stats}
qf {stats}
qbeta(p, shape1, shape2, ncp = 0, lower.tail = TRUE,
log.p = FALSE)
The Beta Distribution
Density, distribution function, quantile function and random generation for the Beta distribution with parameters shape1 and shape2 (and optional non-centrality parameter ncp).
Press F1 for additional help
Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Tab

Go to file/function | Addins | Project: (None)

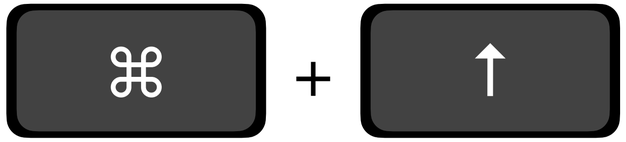
Untitled1* | Source on Save | Run | Source

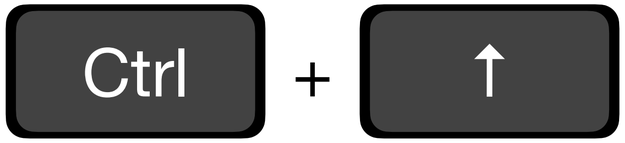
```
1
```

```
qplot(year, -rank, data = james, geom = "line")
qplot(year, -rank, data = james, geom = "line", color = sex)
qplot(year, -rank, data = james_boy, geom = "line")
qplot(x = ano, y = total, data = present, geom = "line")
qplot(x = ano, y = perc, data = present, geom = "line")
qplot(x = ano, y = meninos, data = present, geom = "line") +
qplot(x = ano, y = meninas, data = present, geom = "line")
qplot(ano, meninos, data = arbuthnot, geom = "line")
qplot(ano, total, data = arbuthnot)
qplot(ano, total, data = arbuthnot, geom = "line")
qplot(ano, prop, data = arbuthnot, geom = "line")
qplot(ano, mais_meninos, data = arbuthnot)
```

R Script

```
> q
```





Environment | History | Connections | Tutorial

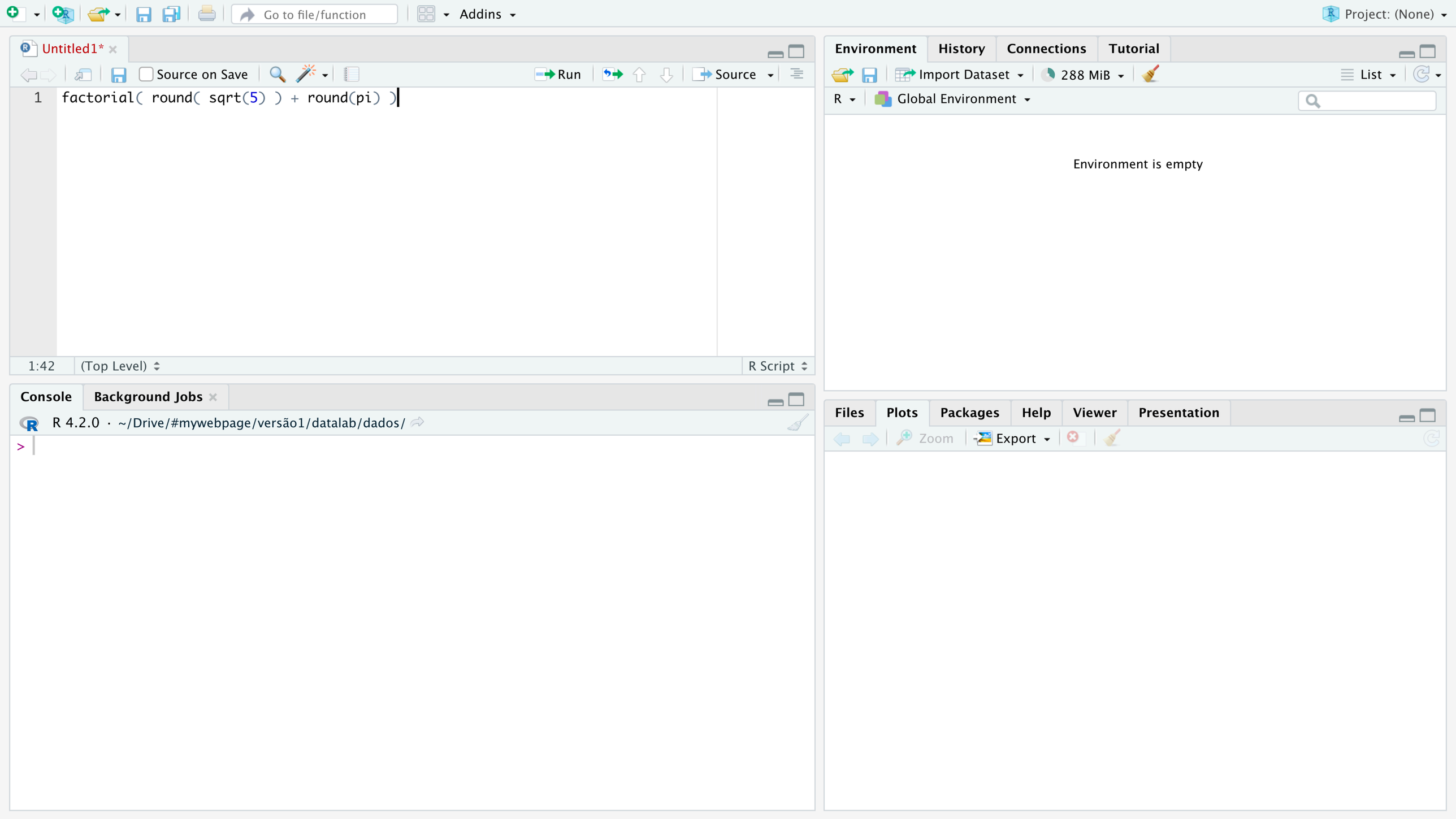
Import Dataset | 291 MiB | List

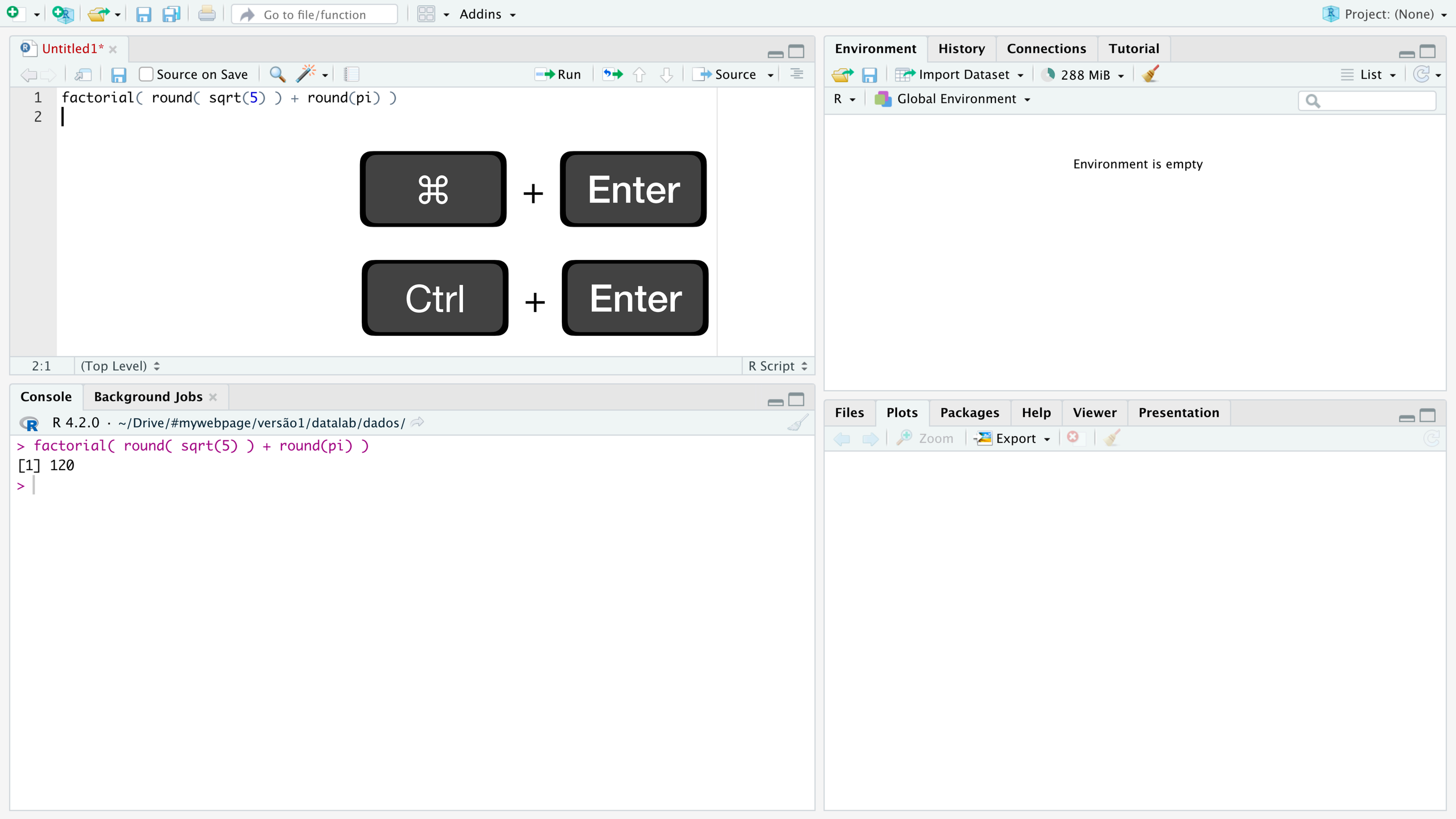
R | Global Environment

Environment is empty

Files | Plots | Packages | Help | Viewer | Presentation

Zoom | Export


Go to file/function
Addins
Project: (None)
Untitled1*
Source on Save
Run
Source
1 factorial(round(sqrt(5)) + round(pi))
1:42
(Top Level)
R Script
Environment
History
Connections
Tutorial
Import Dataset
288 MiB
List
R
Global Environment
Environment is empty
Console
Background Jobs
R 4.2.0 · ~/Drive/#mywebpage/versão1/datalab/dados/
>
Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export


Go to file/function
Addins
Project: (None)
Untitled1*
Source on Save
Run
Source
1 factorial(round(sqrt(5)) + round(pi))
2
⌘ + Enter
Ctrl + Enter
2:1
(Top Level)
R Script
Environment
History
Connections
Tutorial
Import Dataset
288 MiB
List
R
Global Environment
Environment is empty
Console
Background Jobs
R 4.2.0 · ~/Drive/#mywebpage/versão1/datalab/dados/
> factorial(round(sqrt(5)) + round(pi))
[1] 120
>
Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export

Vetores

E operações

Definindo vetores em um objeto

- Como já vimos, podemos criar vetores por meio da função c( )

```
vet <- c(1, 2, 3, 10, 100)
vet
[1] 1   2   3  10 100
```

Operações elemento a elemento

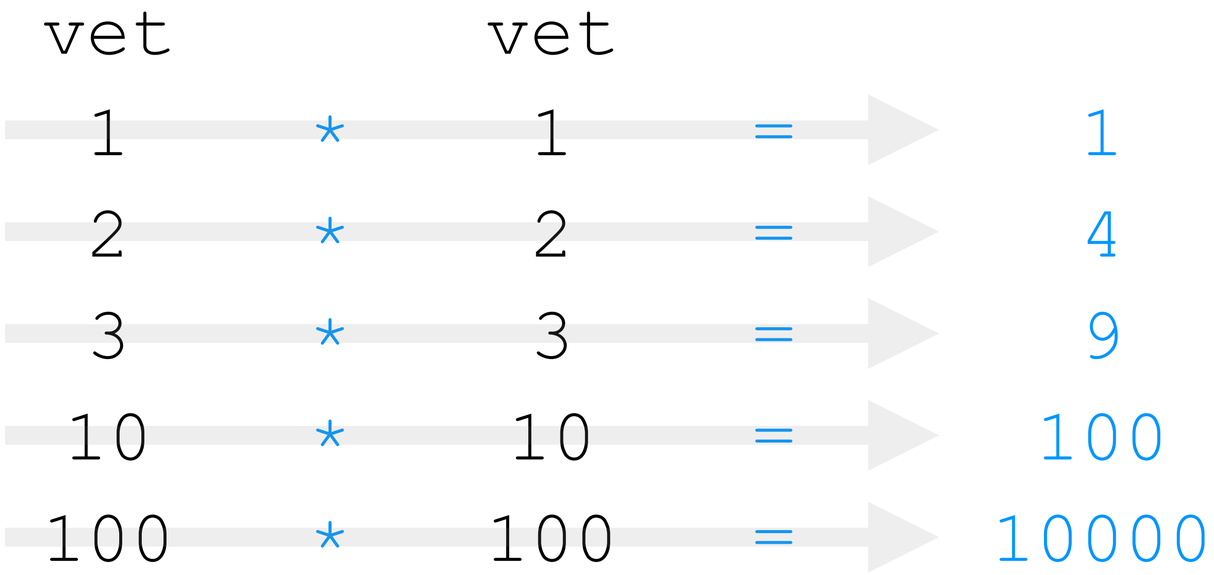
```
vet <- c(1, 2, 3, 10, 100)

vet + 4
[1]  5   6   7  14 104

vet * 4
[1]  4   8  12  40 400
```

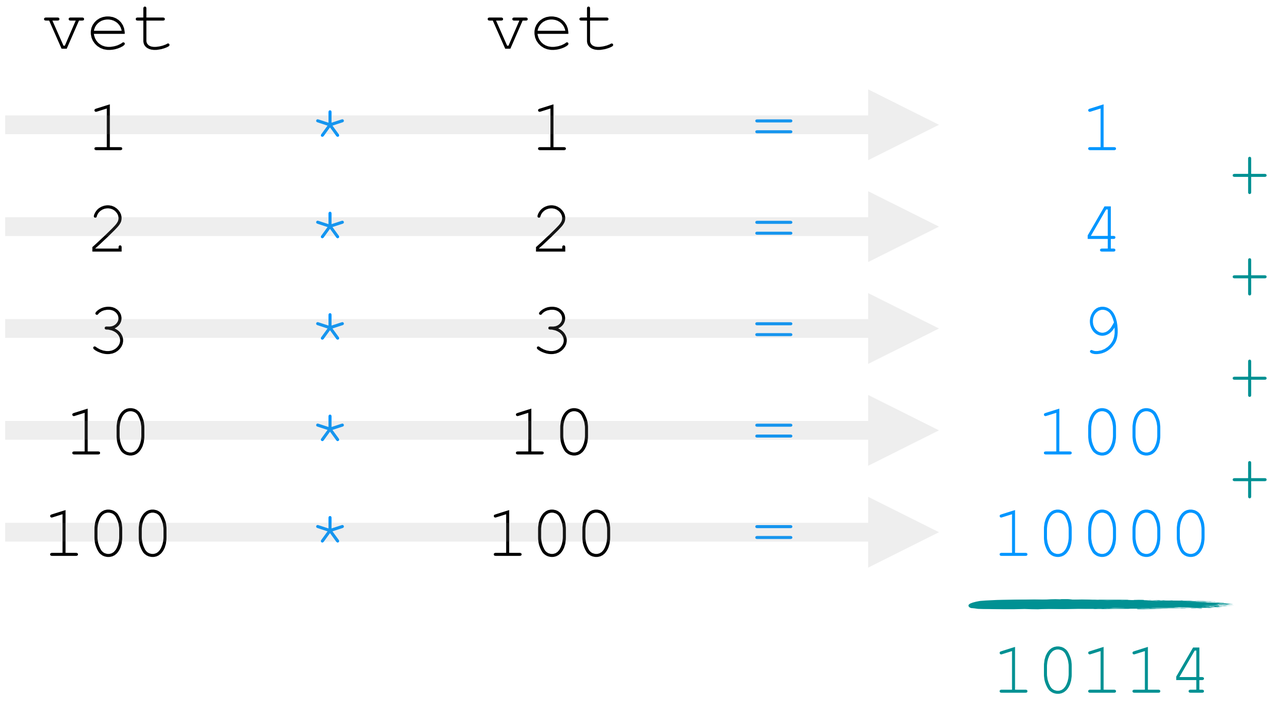
Operações elemento a elemento

```
vet * vet
[1] 1   4   9  100 10000
```



Produto interno

```
vet %*% vet
     [,1]
[1,] 10114
```



Produto externo

```
vet %o% vet
```

```
      [,1] [,2] [,3] [,4]   [,5]
[1,]     1    2    3   10    100
[2,]     2    4    6   20    200
[3,]     3    6    9   30    300
[4,]    10   20   30  100   1000
[5,]   100  200  300 1000 10000
```

Matrizes

E operações

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
```

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
```

Vetor com os elementos da matriz

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
```

Número de linhas da matriz

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 3)
```

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 3)
```

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 3, byrow = T)
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
```

Criando uma matriz

- Podemos criar matrizes por meio da função matrix( )

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 3, byrow = T)
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
```

Definindo uma matriz em um objeto

```
mat <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
mat
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Multiplicando matrizes

```
mat23 <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
mat32 <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 3)
mat22 <- mat23 %*% mat32
mat22
```

```
     [,1] [,2]
[1,]   22   49
[2,]   28   64
```

Transposta e Inversa de uma matriz

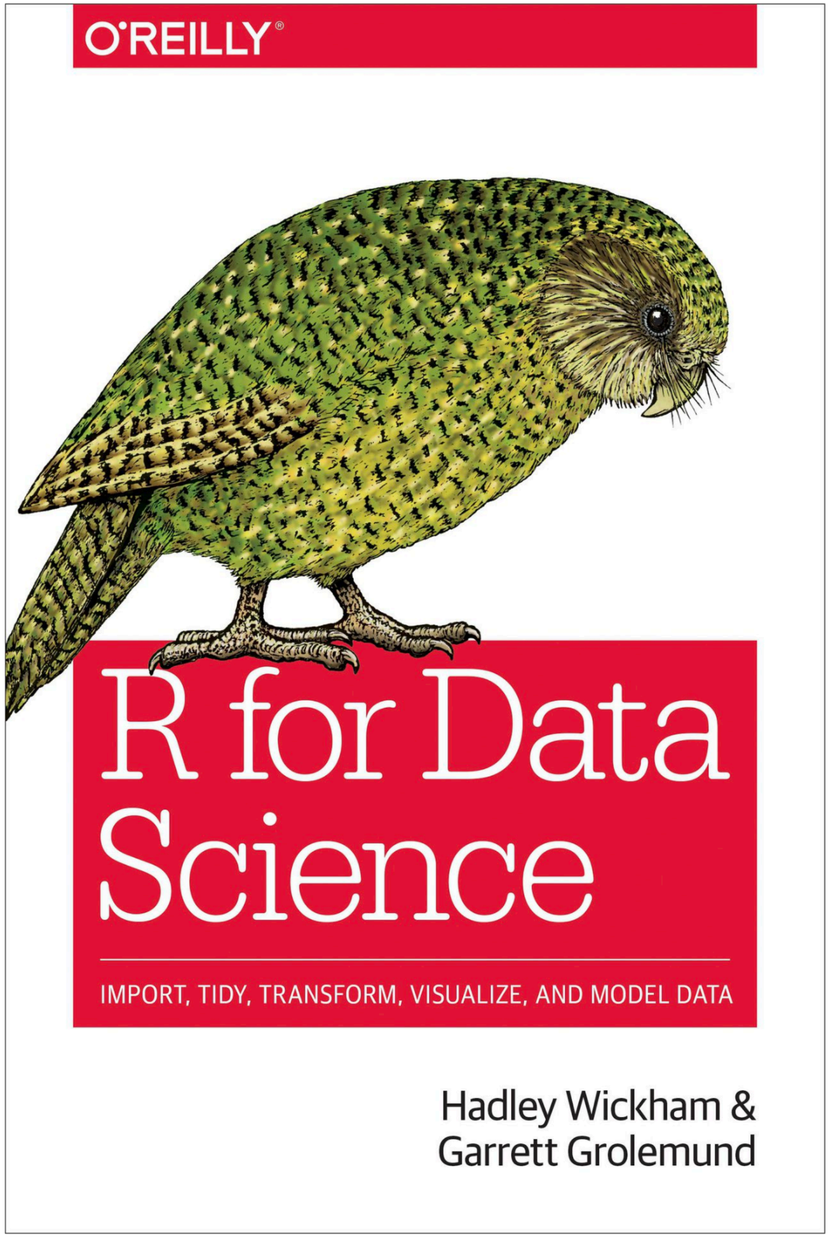
```
t(mat22)
```

```
     [,1] [,2]
[1,]   22   28
[2,]   49   64
```

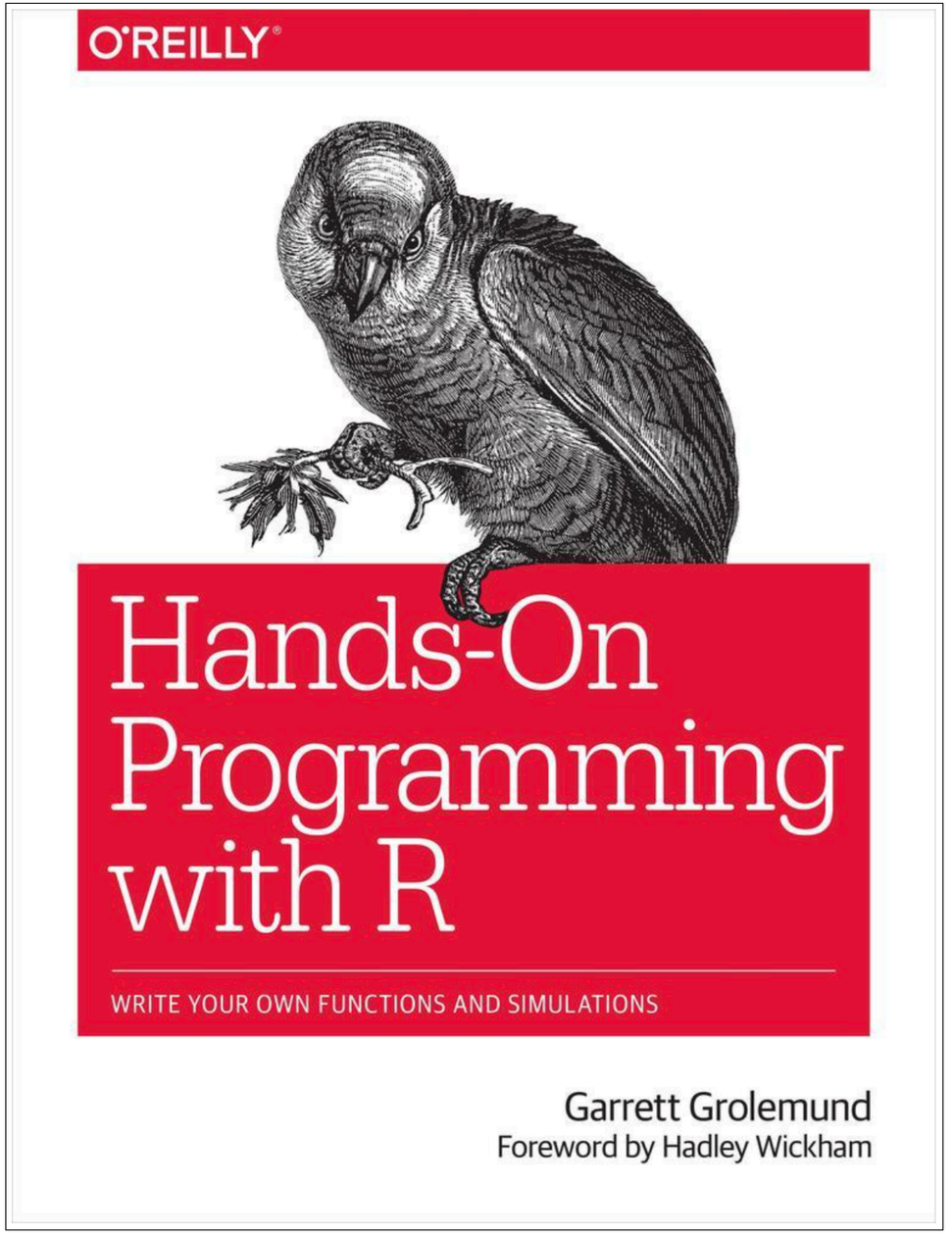
```
solve(mat22)
```

```
           [,1]       [,2]
[1,]  1.7777778 -1.3611111
[2,] -0.7777778  0.6111111
```

Referências



https://r4ds.had.co.nz/



https://rstudio-education.github.io/hopr/